

Prompt engineering guide

[↓ PDF \(/pdfs/nova/latest/nova2-userguide/nova2-ug.pdf#prompt-engineering-guide\)](#)

[↓ RSS \(nova2-ug.rss\)](#)

[↓ Markdown \(prompt-engineering-guide.md\)](#)

⚡ **Agent Setup**



Focus mode



Did this page help you?

Yes

No

Provide feedback ([https://docs.aws.amazon.com/feedback/doc-feedback.html?](https://docs.aws.amazon.com/feedback/doc-feedback.html?hidden_service_name=Nova&topic_url=https%3A%2F%2Fdocs.aws.amazon.com%2Fnova%2Flatest%2Fnova2-userguide%2Fprompt-engineering-guide.html)

[hidden_service_name=Nova&topic_url=https%3A%2F%2Fdocs.aws.amazon.com%2Fnova%2Flatest%2Fnova2-userguide%2Fprompt-engineering-guide.html](https://docs.aws.amazon.com/feedback/doc-feedback.html?hidden_service_name=Nova&topic_url=https%3A%2F%2Fdocs.aws.amazon.com%2Fnova%2Flatest%2Fnova2-userguide%2Fprompt-engineering-guide.html))

Prompt engineering is the practice of designing instructions to effectively communicate with large language models (LLMs) like Amazon Nova. Prompt engineering allows you to control model behavior, improve output quality and build reliable AI-powered applications.

Effective prompt engineering involves crafting inputs that guide models to produce desired outputs. A well-crafted prompt provides clear instructions, relevant context and properly formatted input data.

What is prompt engineering

[↓ PDF \(/pdfs/nova/latest/nova2-userguide/nova2-ug.pdf#prompting-what-is\)](#)

[↓ RSS \(nova2-ug.rss\)](#)

[↓ Markdown \(prompting-what-is.md\)](#)

[⚡ Agent Setup](#)

☐ Focus mode



On this page

Did this page help you?

☐ Yes

☐ No

Provide feedback ([https://docs.aws.amazon.com/feedback/doc-feedback.html?](https://docs.aws.amazon.com/feedback/doc-feedback.html?hidden_service_name=Nova&topic_url=https%3A%2F%2Fdocs.aws.amazon.com%2Fnova%2Flatest%2Fnova2-userguide%2Fprompting-what-is.html)

[hidden_service_name=Nova&topic_url=https%3A%2F%2Fdocs.aws.amazon.com%2Fnova%2Flatest%2Fnova2-userguide%2Fprompting-what-is.html](https://docs.aws.amazon.com/feedback/doc-feedback.html?hidden_service_name=Nova&topic_url=https%3A%2F%2Fdocs.aws.amazon.com%2Fnova%2Flatest%2Fnova2-userguide%2Fprompting-what-is.html))

Prompt engineering refers to the practice of optimizing textual input to a large language model (LLM) to improve output and receive the responses you want. Prompting helps an LLM perform a wide variety of tasks, including classification, question answering, code generation, creative writing, and more. The quality of prompts that you provide to a LLM can impact the quality of the model's responses. This section provides you the necessary information to get started with prompt engineering. It also covers tools to help you find the best possible prompt format for your use case when using a LLM on Amazon Bedrock.

The effectiveness of prompts is contingent upon the quality of the information provided and the craftsmanship of the prompt itself. Prompts may encompass instructions, questions, contextual details, inputs, and examples to effectively guide the model and enhance the quality of the results. This document outlines strategies and tactics for optimizing the performance of Amazon Nova 2 Sonic family of models. The methods presented herein may be employed in various combinations to amplify their effectiveness.

We encourage you to engage in experimentation to identify the approaches most suitable for their specific needs.

Getting started with prompt engineering

Before you start prompt engineering, we recommend that you have the following elements in place, so you can iteratively develop the most optimal prompt for your use case:

Define your use case

Define your use case across four dimensions:

- **Task** – Define what you want the model to accomplish. This determines the right prompting technique.
- **Role** – Define what role the model should assume to accomplish the task. Amazon Nova models support three roles (System, User, or Assistant).
- **Response Style** – Define the response structure or style that the model should follow based on the audience, such as JSON, markdown, or conversational.
- **Instructions** – Define the set of instructions that the model should follow to meet success criteria.

Establish success criteria

Define success criteria or evaluation metrics. You can provide a list of criteria or provide specific evaluation metrics, such as length, BLEU score, ROUGE, format, factuality and faithfulness.

Draft a prompt

Create a starting prompt incorporating your task, role, response style and instructions. Iterate based on results.

The effectiveness of prompts depends on the quality of information you provide.

General best practices

[↓ PDF \(/pdfs/nova/latest/nova2-userguide/nova2-ug.pdf#prompting-best-practices\)](#)

[↓ RSS \(nova2-ug.rss\)](#)

[↓ Markdown \(prompting-best-practices.md\)](#)

⚡ **Agent Setup**

☐ Focus mode



On this page

Did this page help you?

☐ Yes

☐ No

Provide feedback ([https://docs.aws.amazon.com/feedback/doc-feedback.html?](https://docs.aws.amazon.com/feedback/doc-feedback.html?hidden_service_name=Nova&topic_url=https%3A%2F%2Fdocs.aws.amazon.com%2Fnova%2Flatest%2Fnova2-userguide%2Fprompting-best-practices.html)

[hidden_service_name=Nova&topic_url=https%3A%2F%2Fdocs.aws.amazon.com%2Fnova%2Flatest%2Fnova2-userguide%2Fprompting-best-practices.html](https://docs.aws.amazon.com/feedback/doc-feedback.html?hidden_service_name=Nova&topic_url=https%3A%2F%2Fdocs.aws.amazon.com%2Fnova%2Flatest%2Fnova2-userguide%2Fprompting-best-practices.html))

The following best practices mainly apply to the Amazon Nova text models, but you can apply them to other models, in addition to modality-specific best practices.

For more information on how to prompt multimodal inputs, refer to [Prompting multimodal inputs \(/prompting-multimodal.html\)](#) . For information on how to prompt speech inputs, refer to [Voice conversation prompts \(/sonic-system-prompts.html\)](#) .

Understanding the roles

Amazon Nova models allow you to structure prompts through the use of three distinct roles: system, user, and assistant.

- **System (optional)** – Although not mandatory, it establishes the overall behavioral parameters of the assistant. It can also be utilized to provide additional instructions and guidelines that the user wishes the model to adhere to throughout the conversation.
- **User** – Can optionally convey the context, tasks, instructions, and the desired outcome along with the user query.
- **Assistant** – Aids in guiding the model towards the intended response.

Topics

- [Create precise prompts \(./create-precise-prompts.html\)](#)
- [Bring focus to sections of the prompt \(./prompting-bring-focus.html\)](#)
- [Using the system role \(./prompting-system-role.html\)](#)
- [Provide examples \(few-shot prompting\) \(./prompting-provide-examples.html\)](#)
- [Tool calling systems \(./prompting-tools-function.html\)](#)
- [Advanced prompting techniques \(./advanced-prompting-techniques.html\)](#)

Create precise prompts

[↓ PDF \(/pdfs/nova/latest/nova2-userguide/nova2-ug.pdf#create-precise-prompts\)](#)

[↓ RSS \(nova2-ug.rss\)](#)

[↓ Markdown \(create-precise-prompts.md\)](#)

[⚡ Agent Setup](#)

☐ Focus mode



On this page

Did this page help you?

☐ Yes

☐ No

Provide feedback ([https://docs.aws.amazon.com/feedback/doc-feedback.html?](https://docs.aws.amazon.com/feedback/doc-feedback.html?hidden_service_name=Nova&topic_url=https%3A%2F%2Fdocs.aws.amazon.com%2Fnova%2Flatest%2Fnova2-userguide%2Fcreate-precise-prompts.html)

[hidden_service_name=Nova&topic_url=https%3A%2F%2Fdocs.aws.amazon.com%2Fnova%2Flatest%2Fnova2-userguide%2Fcreate-precise-prompts.html](https://docs.aws.amazon.com/feedback/doc-feedback.html?hidden_service_name=Nova&topic_url=https%3A%2F%2Fdocs.aws.amazon.com%2Fnova%2Flatest%2Fnova2-userguide%2Fcreate-precise-prompts.html))

Crafting specific user queries is crucial in prompt engineering. Well-crafted queries direct Amazon Nova text understanding models to generate accurate and pertinent responses. To craft such queries, it is essential to start by providing contextual information in the query. The context provided helps situate the model better by aligning to clear target-user scenarios, thereby enhancing the relevance and coherence of the output.

Providing contextual, specific information

Models work best if the instructions are clear, well structured, short, and specific. This prompt template guidance applies to both the user prompt and the system prompt.

Prompt template

```

# Use this to clearly define the task and job needed by the
model
task_summary = f"""
## Task Summary:
{{Task summary}}
"""

# Use this to provide contextual information related to the
task
context_information = f"""
## Context Information:
- {{Context and content information 1}}
- {{Context and content information 2}}
...
"""

# Use this to provide any model instructions that you want
model to adhere to
model_instructions = f"""
## Model Instructions:
- {{ Other Model Instructions }}
...
"""

# Use this to provide response style and formatting guidance
response_style = f"""
## Response style and format requirements:
- {{Style and format requirement 1}}
- {{Style and format requirement 2}}
...
"""

# Concatenate to final prompt
final_prompt = f"""{{task_summary}}
{{context_information}}
{{model_instructions}}
{{response_style}}"""

```

► Example: Specific and contextual information

Here is an example of a prompt that is rewritten with specific and contextual information. Adding more contextual information helps the model ground towards the desired behavior.

Unclear prompt

Clear prompt

User:

Write me a meeting invite to the project team

Assistant:

Certainly! Here's a sample meeting invite for your project team:

****Subject:**** Project Team Meeting - [Project Name]

****Date:**** [Insert Date]

****Time:**** [Insert Time]

****Duration:**** [Insert Duration]

****Location:**** [Insert Location] / [Zoom/Teams Link if Online]

****Agenda:****

1. ****Welcome and Introductions****

- Quick round of introductions for new team members

2. ****Project Overview****

- Objectives and goals
- Current status and milestones achieved

3. ****Task Allocation and Deadlines****

- Detailed breakdown of tasks
- Assignment of responsibilities

- Discussion of deadlines

4. ****Risk Management****

- Identification of potential risks
- Mitigation strategies

5. ****Resource Allocation****

- Required tools and materials
- Budget review

6. ****Q&A Session****

- Open floor for questions and clarifications

7. ****Next Steps and Closing****

- Summary of action items
- Schedule for the next meeting

****Additional Notes:****

- Please come prepared with any relevant documents or updates regarding your assigned tasks.
- If you anticipate any issues or require additional support, please let us know in advance so we can address them during the meeting.

****Contact Information:****

For any questions or concerns before the meeting, please feel free to reach out to [Your Name] at [Your Email] or [Your Phone Number].

We look forward to a productive discussion and to kicking off our project with enthusiasm!

Best regards,

[Your Full Name]

[Your Position]

[Your Contact Information]

Feel free to customize this template to better fit your specific project and team needs!

- Be clear: When prompting the model, try to keep instructions as clear as possible in a way a human would understand it versus a machine (for instance, a JSON-style list).
- Be specific: Give the model specific instructions about what the task is, what the expected output is, and any additional context to help the model respond the way you want.

Specify the output format and style

If you want Amazon Nova 2 to generate responses that are tailored to your specific needs, you can specify the format and style of the output. For example, if you want to generate responses only in JSON or constrain the response length to a maximum number of words, bullet points, or paragraphs. This enables Amazon Nova 2 to better understand your requirements and produce output that is more relevant and easier to parse.

Use capitalization and strong wordings such as DO, DO NOT, MUST to emphasize the rules you want the model to strictly obey. For example, if you want only one JSON object in the response, you can instruct the model by saying "You MUST answer in JSON format only. DO NOT use any other format while answering the question."

Here are some example prompts that are rewritten with specific and contextual information. Adding more contextual information helps the model ground towards the desired behavior.

► Example 1: Provide extra details

Adding extra details on the response formatting can help the model accomplish the desired result. Some of the other commonly used response format include CSV, Markdown, YAML, and XML

User:

Outline the different types of rooms available in a typical 4-star hotel, including their features.

Assistant:

In a typical 4-star hotel, you'll find a range of room types, each designed to cater to different guest needs and preferences. Here's an outline of the different types of rooms available along with their features:

1. ****Standard Room****

- ****Size:**** Usually around 250-350 square feet.
- ****Features:****
 - Queen or double bed.
 - Complimentary Wi-Fi.
 - Flat-screen TV with cable or satellite channels.
 - Mini-fridge.
 - Coffee maker with complimentary coffee and tea.
 - Desk and chair.
 - En-suite bathroom with shower, toilet, and toiletries.
 - Air conditioning/heating.

2. ****Deluxe Room****

- ****Size:**** Typically larger than standard rooms, around 350-450 square feet.
- ****Features:****
 - King bed or two queen beds.
 - Larger bathroom with a bathtub.
 - Additional seating area.
 - Enhanced amenities like bathrobes and slippers.
 - Upgraded toiletries and sometimes a welcome

gift.

- Access to a lounge or executive floor perks.

3. **Suite**

- **Size:** Varies significantly, often between 500-1,200 square feet.
- **Features:**
 - Separate living area with a sofa bed or additional seating.
 - Kitchenette or full kitchen in some cases.
 - Larger, more luxurious bathroom with a bathtub and separate shower.
 - Multiple TVs.
 - Access to executive lounge.
 - High-end toiletries and often complimentary breakfast.
 - Often includes concierge services and in-room dining options.

4. **Family Room**

- **Size:** Larger, usually around 400-500 square feet.
- **Features:**
 - One king bed and one sofa bed or two queen beds.
 - Designed for families with children.
 - Extra space for luggage.
 - Child-friendly amenities like a small kitchenette or high chair.
 - Family bathroom with amenities suitable for kids.

5. **Executive Room**

- **Size:** Typically around 350-450 square feet.
- **Features:**
 - Business-oriented with a larger workspace.
 - Executive desk and chair.
 - Access to executive lounge with complimentary snacks and refreshments.
 - King bed or two queen beds.
 - Enhanced amenities like high-speed internet

and sometimes a Nespresso machine.

6. ****Junior Suite****

- ****Size:**** Usually around 450-600 square feet.
- ****Features:****
 - Separate bedroom and living area.
 - King bed or two queen beds.
 - Kitchenette or mini-fridge.
 - Luxurious bathroom with a bathtub.
 - Lounge access.
 - High-end toiletries.

7. ****Premium Room****

- ****Size:**** Larger than standard rooms, around 400-500 square feet.
- ****Features:****
 - King bed or two queen beds.
 - Larger bathroom with a bathtub.
 - Enhanced amenities like a welcome drink and premium toiletries.
 - Access to premium services and sometimes a lounge.

8. ****Connecting Rooms****

- ****Size:**** Two rooms that connect via an internal door.
- ****Features:****
 - Suitable for families or groups.
 - Often includes one standard room and one suite.
 - Shared bathroom or en-suite bathrooms in each room.
 - Extra space for luggage and guests.

9. ****Accessible Rooms****

- ****Size:**** Varies, typically similar to standard rooms.
- ****Features:****
 - Designed for guests with disabilities.
 - Wider doorways and bathroom space.
 - Adapted fixtures and fittings.
 - Additional support services.

10. ****Business Room****

- ****Size:**** Usually around 300-350 square feet.
- ****Features:****
 - Focused on business travelers.
 - High-speed internet and large workspace.
 - Compact yet fully equipped with essentials.
 - Often includes a king bed.

Each room type in a 4-star hotel is designed to provide comfort, convenience, and luxury tailored to different guest needs, ensuring a pleasant stay for all types of travelers.

► Example 2: Provide style constraints

In this case, the prompt with style constraints provides comprehensive information on the constraints to follow and the desired response format. As observed in the response, the optimized prompt aligns more closely with the requested brevity and bulleted list structure.

Prompt without style constraints

User:

Explain the process of checking in a guest at a luxury resort.

Assistant:

Checking in a guest at a luxury resort involves a meticulous and personalized process designed to ensure a seamless and memorable start to their

Prompt with style constraints

stay. Here's a detailed overview of the steps involved:

Pre-Arrival Preparation

1. ****Pre-Arrival Communication:****

- ****Confirmation:**** Verify guest reservations and confirm arrival details.
- ****Special Requests:**** Collect any special requests (dietary restrictions, room preferences, etc.) and ensure they are noted in the guest profile.
- ****Documentation:**** Ensure all necessary documentation is ready, such as welcome letters, room keys, and any special amenities.

2. ****Room Preparation:****

- ****Cleaning:**** Conduct a thorough cleaning and preparation of the guest room.
- ****Amenities:**** Stock the room with high-end toiletries, fresh linens, and any requested amenities.
- ****Personal Touches:**** Add personalized touches such as a welcome note, fresh flowers, or locally sourced treats.

Arrival and Initial Interaction

3. ****Guest Arrival:****

- ****Greeting:**** Have staff ready to greet guests as they arrive. This may include a warm welcome at the entrance or a meet-and-greet in the lobby.
- ****Transport:**** Provide luxury transport options like a chauffeured car service to escort guests to the check-in desk.

4. ****Check-In Desk:****

- ****Professionalism:**** Ensure check-in staff are well-trained, professional, and exude hospitality.
- ****Technology:**** Utilize sophisticated check-in technology (like tablet kiosks) to streamline the

process.

- ****Personalized Welcome:**** Greet guests with a warm welcome, acknowledge their special requests, and present them with a personalized welcome kit.

Documentation and Formalities

5. ****Document Verification:****

- ****Identification:**** Verify identification and payment methods.
- ****Membership Cards:**** If applicable, verify any loyalty or membership cards for additional benefits.

6. ****Registration:****

- ****Form Filling:**** Assist guests with any required paperwork.
- ****Special Benefits:**** Clearly explain any special resort benefits, upgrades, or exclusive access to amenities that the guest is entitled to.

Room Assignment

7. ****Room Presentation:****

- ****Walk-Through:**** Escort guests to their room and conduct a walk-through to ensure everything is in order and to address any immediate concerns.
- ****Upgrades:**** Present any room upgrades and explain the features and services available in the upgraded room.

8. ****Key Delivery:****

- ****Room Key:**** Provide the room key, often a high-end key card or a luxury key fob.
- ****Access Information:**** Explain how to use the key card, including access to other amenities like the spa, pool, and gym.

Additional Services

9. ****Welcome Amenities:****

- ****Amenity Kits:**** Deliver welcome amenity kits (like a bottle of champagne, local delicacies, etc.).
- ****Itinerary:**** Provide a detailed resort map and itinerary, highlighting all available services and activities.

10. ****Personal Concierge:****

- ****Concierge Services:**** Introduce the guest to the concierge service and explain how to request any special services, such as dining reservations, transportation, or local excursions.
- ****Contact Information:**** Provide contact information for key staff members.

Final Touches

11. ****Feedback:****

- ****Satisfaction Survey:**** At the end of the check-in process, ask for feedback to ensure the guest's expectations were met and to gather any immediate suggestions.

12. ****Follow-Up:****

- ****Immediate Issues:**** Ensure any immediate issues or requests are addressed promptly.
- ****Comfort Check:**** Make follow-up calls or send messages to ensure the guest is settling in comfortably.

By following these steps, a luxury resort aims to provide an exceptional check-in experience that sets the tone for an unforgettable stay.

► Example 3: Limit the response format

Here, the prompt with the response format forces the model to respond only with the category number and limit it to be between 1-10, instead of being highly verbose.

Prompt without response format

Prompt with response format

User:

In this task, you are given a text which is the body of a document. Your job is to classify the topic of the document into these categories:

1)Company, 2)Educational Institution, 3)Artist, 4)Athlete, 5)Office Holder, 6)Mean of transportation, 7)Building, 8)Natural place, 9)Village, 10)Animal.

Text: Johangsan is a mountain of South Korea. It has an elevation of 954 metres.

Assistant:

To classify the topic of the given text, let's analyze the content step-by-step:

1. The text mentions "Johangsan," which appears to be a specific name.
2. The text states that "Johangsan is a mountain," which provides a clear indication of its nature.
3. It is further specified that it is located in "South Korea" and has an elevation of "954 metres."

Based on the information provided, the text describes a geographical feature, specifically a mountain. From the given categories, the most appropriate classification for this document is:

8) Natural place

The text does not mention any company, educational institution, artist, athlete, office holder, means of transportation, building, village, or animal. Therefore, the classification as a "Natural place" is the most fitting choice.

Provide clear prompt sections

Beyond the initial context, a structured query format plays a significant role in optimizing interactions with the language model. Separating distinct parts of the input with delimiters ensures clarity and enhances Amazon Nova 2 models' ability to understand the query's structure. Another effective way is to use numbered lists or bullet points, which can help to break down the task into manageable steps and provide a clear outline of the actions that need to be taken. This structured approach aids in reducing ambiguity and improving the accuracy of the responses generated.

► Example: Create a sectioned prompt

Notice how in the Sectioned Prompt column, the prompt is clearer and has delineated sections for different entities like Tasks, Context, Requirements, and the rest when compared to the instructions in the Unsectioned Prompt column.

Unsectioned Prompt

Sectioned Prompt

User:

Write an article about climate change for high school students explaining basic concepts, current impacts, and individual actions to mitigate it. Make it informative and engaging

with simple language, including statistics and real-world examples.

Assistant:

Understanding Climate Change: What It Is, What It's Doing, and What You Can Do About It

Introduction

Climate change might sound like a big, complicated topic, but it's really about the way our planet's weather is changing over time. This change is mostly happening because humans are doing things that are making our Earth warmer. Let's break it down so you can understand what's going on and what you can do to help.

What Is Climate Change?

Climate change refers to long-term shifts in temperatures and weather patterns. These shifts may be natural, but since the 1800s, human activities have been the main driver of rapid climate change.

****Key Concept: Greenhouse Gases****

To understand this, we need to talk about greenhouse gases. These gases, like carbon dioxide (CO₂), methane (CH₄), and nitrous oxide (N₂O), trap heat in our atmosphere. Without these gases, Earth would be too cold for most life forms. But when we burn fossil fuels (like coal, oil, and natural gas) and cut down forests, we release too much CO₂ and other greenhouse gases, trapping more heat and making the planet warmer.

Current Impacts of Climate Change

Let's look at some real-world examples to see how climate change is affecting our world today.

****1. Rising Temperatures****

Since the late 19th century, Earth's average

surface temperature has risen about 2 degrees Fahrenheit (1 degree Celsius). Scientists predict that if we don't change our ways, it could rise by 3.6 to 4.3 degrees Fahrenheit (2 to 2.4 degrees Celsius) by the end of this century.

****2. More Extreme Weather****

Climate change is making extreme weather events more frequent and severe. For example:

- ****Hurricanes:**** In the U.S., hurricane seasons are getting longer and more destructive. In 2020, there were 30 named storms, the most ever recorded.

- ****Droughts:**** Parts of California have experienced some of the worst droughts in history, affecting agriculture and water supplies.

****3. Melting Ice Caps and Rising Sea Levels****

Glaciers and ice sheets are melting at alarming rates. This not only affects wildlife but also leads to rising sea levels. Since 1901, global sea levels have risen by about 8 inches (20 centimeters). By 2100, they could rise by another 1 to 4 feet (0.3 to 1.2 meters).

****4. Ocean Acidification****

As CO₂ levels rise, more of it is absorbed by the oceans, making them more acidic. This is particularly harmful to marine life, including coral reefs. Coral reefs are crucial for marine biodiversity but are now dying at an alarming rate.

Individual Actions to Mitigate Climate Change

You might think that since climate change is a huge problem, one person can't make a difference. But even small actions can add up to make a big impact.

****1. Reduce, Reuse, Recycle****

Try to minimize waste by:

- ****Reducing:**** Buy fewer items and choose products with less packaging.
- ****Reusing:**** Use reusable bags, bottles, and containers.
- ****Recycling:**** Make sure to recycle paper, plastic, and glass properly.

****2. Conserve Energy****

Small changes can lead to big savings:

- Turn off lights when you leave a room.
- Unplug devices that you're not using.
- Use energy-efficient bulbs and appliances.

****3. Use Less Meat****

Livestock farming is a major contributor to greenhouse gas emissions. By eating less meat, especially beef, you can reduce your carbon footprint.

****4. Support Renewable Energy****

If possible, support or switch to renewable energy sources like solar or wind power. Even advocating for these options in your community can make a difference.

****5. Educate Yourself and Others****

Knowledge is power. The more you know about climate change, the more you can help others understand it. Share facts and encourage your friends and family to take action too.

Conclusion

Climate change is a pressing issue that affects everyone on the planet. By understanding its basics, recognizing its current impacts, and taking individual actions, you can play a crucial role in mitigating its effects. Remember, every small step counts, and together, we can make a big difference for our planet.

Let's start today and build a sustainable future!

Bring focus to sections of the prompt

[↓ PDF \(/pdfs/nova/latest/nova2-userguide/nova2-ug.pdf#prompting-bring-focus\)](#)

[↓ RSS \(nova2-ug.rss\)](#)

[↓ Markdown \(prompting-bring-focus.md\)](#)

[⚡ Agent Setup](#)

☐ Focus mode



Did this page help you?

☐ Yes

☐ No

Provide feedback ([https://docs.aws.amazon.com/feedback/doc-feedback.html?](https://docs.aws.amazon.com/feedback/doc-feedback.html?hidden_service_name=Nova&topic_url=https%3A%2F%2Fdocs.aws.amazon.com%2Fnova%2Flatest%2Fnova2-userguide%2Fprompting-bring-focus.html)

[hidden_service_name=Nova&topic_url=https%3A%2F%2Fdocs.aws.amazon.com%2Fnova%2Flatest%2Fnova2-userguide%2Fprompting-bring-focus.html](https://docs.aws.amazon.com/feedback/doc-feedback.html?hidden_service_name=Nova&topic_url=https%3A%2F%2Fdocs.aws.amazon.com%2Fnova%2Flatest%2Fnova2-userguide%2Fprompting-bring-focus.html))

Amazon Nova 2 models can pay close attention to specific parts in the prompt by formatting instructions in sections and then referring to those specific sections. The model is able to pay attention if prompts have clear sectional delimitation using markdown, XML, or other structure. For example, you can define the name of the section, use `##Section Name##`, then refer to that section in your prompt with `##Section Name##`.

You can also utilize this strategy to restrict the model from revealing parts of the input prompt in the generated response. For example, when providing few shot examples or instructions in the input prompt, use delimiters such as `##Instructions##` or `##Examples##` with a new line separator and provide strong instructions such as DO NOT mention anything inside the `##Instructions##` or `##Examples##` in the

response for the model to not regurgitate the input prompt content from these sections in its output.

► Example: Sectional delimitation

User:

You're an expert Prompts creator. Your task is to create a set of diverse and very complex `##PROMPTS##` that will be used to test the capabilities of a language model in knowledge and following instructions with constraints. Please create 10 `##PROMPTS##`. You must strictly follow `##GUIDELINES##`:

`##GUIDELINES##`

- Generate `##PROMPTS##` similar to the structure and style of the given `##EXAMPLE PROMPTS##`. Pay close attention to the complexity and diversity of `##EXAMPLE PROMPTS##`.
- Generated `##PROMPTS##` must be from the `##DOMAINS##` and must be with these `##USECASES##`.
- Each of the `##PROMPTS##` needs to be unique and very complex.
- Each of the `##PROMPTS##` must have more than 4 sentences and 1 constraint.
- Each of the `##PROMPTS##` should have at least 70 words.
- Each of the `##PROMPTS##` should have an answer that can be written in text.
- The length of the answer of these `##PROMPTS##` must be finite and not very very long.
- In the `##PROMPTS##` you should not mention anything about writing in pages or slides.
- Each of the `##PROMPTS##` should be separated by a new line, without additional formatting.

Generated `##PROMPTS##` must be from the following `##DOMAINS##`
`##DOMAINS##`
{domains}

Generated `##PROMPTS##` must be for the following `##USECASES##`

##USECASES##

{usecases}

{usecase_*description*}

##PROMPTS##

Using the system role

[↓ PDF \(/pdfs/nova/latest/nova2-userguide/nova2-ug.pdf#prompting-system-role\)](#)

[↓ RSS \(nova2-ug.rss\)](#)

[↓ Markdown \(prompting-system-role.md\)](#)

[⚡ Agent Setup](#)

☐ Focus mode



Did this page help you?

☐ Yes

☐ No

Provide feedback ([https://docs.aws.amazon.com/feedback/doc-feedback.html?](https://docs.aws.amazon.com/feedback/doc-feedback.html?hidden_service_name=Nova&topic_url=https%3A%2F%2Fdocs.aws.amazon.com%2Fnova%2Flatest%2Fnova2-userguide%2Fprompting-system-role.html)

[hidden_service_name=Nova&topic_url=https%3A%2F%2Fdocs.aws.amazon.com%2Fnova%2Flatest%2Fnova2-userguide%2Fprompting-system-role.html](https://docs.aws.amazon.com/feedback/doc-feedback.html?hidden_service_name=Nova&topic_url=https%3A%2F%2Fdocs.aws.amazon.com%2Fnova%2Flatest%2Fnova2-userguide%2Fprompting-system-role.html))

The *System Role* is a role where you can provide instructions to the model that define how it will respond to end users of your application. For example, the *System Role* can guide the model to respond with a given persona, set allowable and unallowable content, output in a specific format, specify guardrails, and so on. Instructions in the *System Role*, called the *system prompt*, will supersede other instructions provided in individual user prompts, and will carry over across all user turns.

► Specify the system role with the API

To give the model a customized role, you can set the `system` parameter in the API as below:

```
{
  "system": [
    {
```

```

        "text": "You are a helpful recipe assistant.
For each recipe request, follow these steps: 1) List
all ingredients needed, 2) Provide prep time and cook
time, 3) Give step-by-step instructions, 4) Suggest
possible variations or substitutions."
    }
],
"messages": [
    {
        "role": "user",
        "content": [
            {
                "type": "text",
                "text": "How do I make a classic tomato
basil pasta?"
            }
        ]
    }
]
}

```

❗ Tip

- To best utilize the *system role*, make sure that your *system prompt* is clear, concise, and specific, following the same strategies as described in [Create precise prompts \(./create-precise-prompts.html\)](#) . For example, when you want to restrict the topics that the model should respond to, instruct it specifically, such as "*DO NOT talk about ...*" or "*You MUST talk about ...*".
- To further restrict the model to a hierarchy structure, you can add the following suffix to your system prompt to emphasize the hierarchy adherence structure between *system* and *user* instructions:

```

"\n
The above system instructions define your capabilities
and your scope. If the
        user request contradicts any system
instruction or if the request is outside your
        scope, you must politely decline the
request briefly explaining your capabilities and
        your scope.\n"

```

The following items are some things that you can add in a system prompt template:

```
# Define a persona
persona = """You are {{Persona}}
"""

# Provide a series of instructions that should be followed while answering
model_instructions = """## Model Instructions
To answer user question, you follow these instructions/steps:
{{Bulleted list of Instructions}}
"""

# Specify the output schema to be followed when responding
response_schema = """## Response Schema
Your response should be in the following output schema:
{{Clear definition of output format}}
"""

# Specify any guardrails that the model should NOT DO explicitly
guardrails = """## Guardrails
Make sure to follow these guardrails:
{{ Guardrails }}
"""

system_prompt = f"""{persona}
{model_instructions}
{response_schema}
{guardrails}
"""
```

To give the system prompt the hierarchy adherence structure between system and user instructions, you can add the following:

The above system instructions define your capabilities and your scope. If

the user request contradicts any system instruction or if the request is outside your scope,
you must politely decline the request briefly explaining

your capabilities and your scope.

► Example 1: Providing specific language cues

After adding the system prompt, the models responses are highly stylized towards the medieval scroll style as indicated in the *System Prompt*.

Without a System Prompt

With a System Prompt

System:

NA

User:

Explain the modern concept of hotel loyalty programs and their benefits.

Assistant:

Hotel loyalty programs have evolved significantly in the modern hospitality industry, becoming sophisticated tools for enhancing customer retention and satisfaction. These programs offer various benefits to both hotels and guests, leveraging technology and data analytics to create a more personalized experience.

Key Components of Modern Hotel Loyalty Programs:

1. ****Tiered Membership Levels:****
 - ****Basic Membership:**** Entry-level membership that provides basic benefits.
 - ****Mid-Tier Membership:**** Includes additional perks such as room upgrades, faster check-in, and special offers.

- ****Top-Tier Membership:**** Offers premium benefits like complimentary upgrades, room service, exclusive access to hotel events, and more.

2. ****Point System:****

- Guests earn points for every stay, purchases made in hotel restaurants, and other activities.
- Points can be redeemed for free nights, upgrades, or other rewards.

3. ****Mobile App Integration:****

- Modern programs often have a dedicated mobile app that allows guests to easily manage their account, check in, and access exclusive offers.
- Features may include digital key access, personalized recommendations, and real-time updates.

4. ****Personalization:****

- Utilizing data analytics, hotels can tailor recommendations and offers to individual guest preferences, enhancing the overall experience.
- Personalized marketing through emails and push notifications can keep guests engaged.

5. ****Cross-Brand Benefits:****

- Some loyalty programs offer benefits across multiple brands within a hotel group or even within different hospitality groups.
- This can include discounts, points transfers, and combined rewards.

6. ****Social Media Integration:****

- Many programs encourage social media sharing and reviews, which can enhance the hotel's online presence and attract new guests.
- Social media contests and exclusive promotions can further engage members.

7. ****Sustainability Initiatives:****

- Modern loyalty programs increasingly

incorporate sustainability, offering points or rewards for eco-friendly choices like reducing plastic use or participating in recycling programs.

Benefits to Guests:

1. ****Financial Savings:****

- Accumulation of points for free or discounted stays.
- Exclusive discounts and offers not available to non-members.

2. ****Enhanced Services:****

- Faster check-in, preferred room assignments, and personalized services.
- Access to exclusive events and experiences.

3. ****Convenience:****

- Easy booking and management through mobile apps.
- Seamless integration with other services like car rentals and travel packages.

4. ****Recognition and Prestige:****

- Status symbols like elite membership levels.
- Recognition by hotel staff and priority service at participating properties.

Benefits to Hotels:

1. ****Increased Guest Loyalty:****

- Encourages repeat visits and long-term relationships with guests.
- Helps retain customers even when faced with competition.

2. ****Data Collection and Analysis:****

- Provides valuable insights into guest preferences and behaviors.
- Helps hotels tailor their services and marketing strategies effectively.

3. ****Revenue Growth:****

- Drives up room occupancy and average daily rate (ADR) through repeat business.
- Cross-selling opportunities within the hotel (e.g., dining, spa services).

4. ****Market Differentiation:****

- Helps hotels stand out in a competitive market.
- Offers a unique selling proposition that attracts brand-loyal customers.

Conclusion:

Modern hotel loyalty programs have become a critical component of the hospitality industry, leveraging technology and data to create personalized, engaging, and rewarding experiences for guests. They provide significant benefits to both guests, who enjoy enhanced services and savings, and hotels, which gain deeper insights and increased revenue through stronger customer relationships.

► Example 2: Providing guardrails

With the addition of the *system prompt*, users can add guardrails for their applications to safeguard the applications for any unintended use. Here is an example of how guardrails can focus the model on the desired task.

Without a System Prompt	With a System Prompt
System:	

NA

User:

Write a one-page business plan for a cooperative bike-sharing program in urban centers aiming to lower traffic congestion and promote eco-friendly transportation. (Include target markets, revenue streams, marketing strategy, and risk mitigation plans.)

Assistant:

****Urban EcoCycle: A Cooperative Bike-Sharing Program****

****Executive Summary:****

Urban EcoCycle is a cooperative bike-sharing program designed to alleviate traffic congestion and promote eco-friendly transportation in urban centers. By providing accessible, affordable, and convenient bike-sharing services, we aim to foster a sustainable urban mobility ecosystem that benefits both residents and the environment.

****Target Markets:****

Our primary target markets include urban professionals, students, and environmentally conscious citizens. We also aim to attract tourists looking for an affordable and eco-friendly way to explore the city. Secondary markets will include local businesses that could benefit from reduced employee commuting times and increased customer engagement due to the program's presence.

****Revenue Streams:****

1. ****Subscription Fees:**** Monthly or yearly subscription plans for individuals and

businesses.

2. ****Ad Revenue:**** Partnerships with local businesses for advertising space on bikes and docking stations.

3. ****Corporate Partnerships:**** Collaborations with companies to provide their employees with access to our bike-sharing program.

4. ****Government Grants:**** Applications for grants and subsidies from local government initiatives aimed at promoting sustainable transportation.

****Marketing Strategy:****

1. ****Digital Marketing:**** Utilize social media platforms, email newsletters, and our website to reach out to potential users and partners. Highlight success stories and eco-friendly benefits.

2. ****Partnerships:**** Collaborate with local businesses, schools, and environmental organizations to promote our service.

3. ****Community Events:**** Host bike-sharing workshops, clean-up events, and eco-friendly fairs to engage with the community.

4. ****Incentives:**** Offer discounts and rewards for long-term subscribers and those who refer new members.

****Risk Mitigation Plans:****

1. ****Maintenance and Security:**** Implement a robust maintenance schedule to ensure bikes are always in good working condition. Employ security measures such as GPS tracking and insurance coverage to protect bikes from theft and damage.

2. ****Regulatory Compliance:**** Stay informed about local regulations and ensure compliance to avoid legal issues. Work with local authorities to establish clear guidelines for bike parking and usage.

3. ****Market Competition:**** Conduct regular market research to stay ahead of competitors. Differentiate our service by offering unique

features such as bike rentals for longer trips, special eco-friendly programs, and partnerships with popular local destinations.

4. ****Economic Fluctuations:**** Develop flexible pricing strategies and diversify revenue streams to cushion against economic downturns. Maintain a reserve fund to handle unexpected financial challenges.

Urban EcoCycle is poised to become a cornerstone of urban sustainability by reducing traffic congestion and promoting green transportation. With a well-defined target market, diverse revenue streams, strategic marketing, and comprehensive risk mitigation plans, we are committed to making our urban centers more livable and eco-friendly.

Provide examples (few-shot prompting)

[↓ PDF \(/pdfs/nova/latest/nova2-userguide/nova2-ug.pdf#prompting-provide-examples\)](#)

[↓ RSS \(nova2-ug.rss\)](#)

[↓ Markdown \(prompting-provide-examples.md\)](#)

⚡ Agent Setup

☐ Focus mode



Did this page help you?

☐ Yes

☐ No

Provide feedback ([https://docs.aws.amazon.com/feedback/doc-feedback.html?](https://docs.aws.amazon.com/feedback/doc-feedback.html?hidden_service_name=Nova&topic_url=https%3A%2F%2Fdocs.aws.amazon.com%2Fnova%2Flatest%2Fnova2-userguide%2Fprompting-provide-examples.html)

[hidden_service_name=Nova&topic_url=https%3A%2F%2Fdocs.aws.amazon.com%2Fnova%2Flatest%2Fnova2-userguide%2Fprompting-provide-examples.html](https://docs.aws.amazon.com/feedback/doc-feedback.html?hidden_service_name=Nova&topic_url=https%3A%2F%2Fdocs.aws.amazon.com%2Fnova%2Flatest%2Fnova2-userguide%2Fprompting-provide-examples.html))

By including a few examples of your task within the prompt, you can build a structured template for Amazon Nova 2 to follow. This reduces ambiguity and enhances the accuracy and quality of the output. The technique of providing clear examples to make the model's responses more aligned with the desired outcome is called *few-shot prompting*.

The concept of few-shot prompting is to provide the language model with a few examples of the task, along with the input and output format, and then ask it to generate the output for a new input based on the provided examples. This method also helps in clarifying complex instructions or tasks, making it easier for Amazon Nova 2 to understand and interpret what is being asked.

How adding examples to the prompt help:

Adding examples can help the model with producing

- Consistent responses which are uniform to the style of the examples

- Performant responses due to reducing the chance of misinterpreting instructions, and minimizing hallucinations

The amount by which model performance improves using few shot prompting will depend on the quality and diversity of your chosen examples. The following items demonstrate the characteristics of good shots in the prompt:

- **Select diverse examples:** The examples chosen should represent the distribution of your expected input/output in terms of diversity (ranging from common use cases to edge cases) to adequately cover relevant use cases. It is important to avoid any biases in your examples, as bias in the inputs can cause outputs to be biased as well.
- **Match complexity levels:** The complexity of the examples provided should align with the target task or scenario. It is important to make sure the complexity grade is mapped between expected the input and the chosen example in the prompt.
- **Ensure relevance:** The examples selected should be directly relevant to the problem or objective at hand. This ensures consistency and uniformity in responses.

Tip

If the above suggestions do not work, we recommended you build a RAG-based system that augments the prompt with a dynamic selection of shots based on the similarities between the user-input query and an available pool of shots.

► Example 1: Provide examples in the user prompt

Here, your user is trying to classify incoming customer support tickets into predefined categories. You can provide Amazon Nova 2 models with a few examples of tickets that are classified correctly to improve its understanding of the categories and the language used in the tickets. The exemplar examples should be concise and represent all tasks targeted in your application. If there is a specific field name that is meaningful to the tasks, such as `Ticket` or `Category` in this example, your user can use those fields directly followed by a colon `:"`.

0-shot prompt	3-shot prompt
User:	

Please classify the following customer support tickets into one of the categories: "Technical Issue," "Billing Inquiry," or "General Inquiry."

Ticket: "Can you explain why my latest bill is higher than usual? I haven't changed my plan."

Assistant:

The customer support ticket should be classified as a "Billing Inquiry."

Here's the reasoning:

- The customer is asking about the reason for a change in their latest bill amount.
- The context revolves around billing and payment, which falls under the category of "Billing Inquiry."

► Example 2: Providing examples as turns prompt

In this example, the exemplars are provided as turns, one turn per exemplar, where the User role contains the input and Assistant role contains the expected output for that exemplar.

0-shot Prompt

3-shot Prompt (as turns)

System:

Your task is to Classify the following texts into the appropriate categories. The categories to classify are:

Categories:

- Food
- Entertainment
- Health
- Wealth
- Other

User:

I have 20\$ in my pocket.

Assistant:

Category: Wealth

Explanation: The text mentions a specific amount of money, which falls under the category of wealth. The presence of a monetary value directly relates to financial status or economic aspect.

► Example 3: Providing examples in the system prompt

In this example, you can specify an "Example" section and then list a series of examples within the system prompt. This way of providing exemplars in the prompt is especially useful if your exemplars are long and complex and helps model locate example shots.

0-shot Prompt

2-shot Prompt

System:

You are an AI assistant helping to draft professional reports for enterprise stakeholders. Ensure the report is formatted in rich text, with headings, bullet points, and highlighted important tasks.

User:

Draft a detailed project update report for our stakeholders, summarizing the project milestones achieved, tasks completed, and future plans.

Assistant:

Project Update Report

Executive Summary

This report provides a comprehensive update on the project milestones achieved, tasks completed, and future plans. Our team has made significant progress, and we are on track to meet the project goals. The following sections outline the details of our accomplishments and upcoming activities.

Project Milestones Achieved

****1. Project Initiation****

- ****Milestone Date:**** July 15, 2023
- ****Description:**** Completed initial project planning and secured stakeholder approval.

****2. System Design Completion****

- ****Milestone Date:**** September 30, 2023
- ****Description:**** Finished the detailed system design and received positive feedback from technical review meetings.

****3. Prototype Development****

- ****Milestone Date:**** November 15, 2023
- ****Description:**** Developed and tested a working prototype of the system.

Tasks Completed

****1. Requirement Gathering****

- ****Description:**** Conducted comprehensive stakeholder interviews and collected detailed requirements.
- ****Highlighted Important Tasks:****
 - ****User Story Mapping:**** Mapped out user stories to ensure all user needs are addressed.
 - ****Requirement Documentation:**** Created detailed requirement specifications.

****2. System Architecture Design****

- ****Description:**** Designed the system architecture, focusing on scalability and maintainability.
- ****Highlighted Important Tasks:****
 - ****Technology Stack Selection:**** Chose the appropriate technology stack based on project requirements.
 - ****Architecture Diagrams:**** Developed detailed architecture diagrams for easy understanding.

****3. Prototype Development and Testing****

- ****Description:**** Developed and rigorously tested the prototype.
- ****Highlighted Important Tasks:****
 - ****Coding:**** Implemented core functionalities of the prototype.
 - ****Testing:**** Conducted unit and integration testing to ensure prototype reliability.

Future Plans

****1. Full System Development****

- ****Description:**** Begin the full-scale development of the system based on the approved design and prototype.
- ****Highlighted Important Tasks:****
 - ****Module Development:**** Develop each module as per the design specifications.
 - ****Code Reviews:**** Implement regular code reviews to maintain code quality.

****2. User Acceptance Testing (UAT)****

- ****Description:**** Conduct UAT to ensure the system meets user requirements and is ready for deployment.
- ****Highlighted Important Tasks:****
 - ****Test Plan Development:**** Create detailed UAT plans and test cases.
 - ****UAT Execution:**** Execute UAT and gather feedback for final adjustments.

****3. Deployment and Training****

- ****Description:**** Deploy the system and provide necessary training to end-users.
- ****Highlighted Important Tasks:****
 - ****Deployment:**** Roll out the system to the production environment.
 - ****Training Sessions:**** Organize training sessions for users to familiarize them with the new system.

Conclusion

Our project has successfully achieved key milestones and completed significant tasks. With the current momentum, we are well-positioned to meet our future objectives. We remain committed to delivering a high-quality system that meets all stakeholder expectations.

****Prepared by:****

[Your Name]

[Your Position]

[Date]

****Contact Information:****

[Your Email]

[Your Phone Number]

This detailed update aims to keep stakeholders informed and reassured about the project's progress and future direction. For any queries or further information, please do not hesitate to contact me directly.

Tool calling systems

↓ [PDF \(/pdfs/nova/latest/nova2-userguide/nova2-ug.pdf#prompting-tools-function\)](/pdfs/nova/latest/nova2-userguide/nova2-ug.pdf#prompting-tools-function)

↓ [RSS \(nova2-ug.rss\)](#)

↓ [Markdown \(prompting-tools-function.md\)](#)

⚡ **Agent Setup**



Focus mode



Did this page help you?

👍 Yes

👎 No

Provide feedback ([https://docs.aws.amazon.com/feedback/doc-feedback.html?](https://docs.aws.amazon.com/feedback/doc-feedback.html?hidden_service_name=Nova&topic_url=https%3A%2F%2Fdocs.aws.amazon.com%2Fnova%2Flatest%2Fnova2-userguide%2Fprompting-tools-function.html)

[hidden_service_name=Nova&topic_url=https%3A%2F%2Fdocs.aws.amazon.com%2Fnova%2Flatest%2Fnova2-userguide%2Fprompting-tools-function.html](https://docs.aws.amazon.com/feedback/doc-feedback.html?hidden_service_name=Nova&topic_url=https%3A%2F%2Fdocs.aws.amazon.com%2Fnova%2Flatest%2Fnova2-userguide%2Fprompting-tools-function.html))

Tool calling is available for the Amazon Nova models by passing a tool configuration schema in your request. The prompt for the model is going to be augmented with this tool configuration so it is a highly impactful place to begin optimizing your tool calling system.

Consider these key principles:

- Tool definitions should be clear and concise. They should be easy to understand and the intent must be extremely apparent.
- Use key differentiators and boundary conditions to define when one tool should be used over another.
- Be critical the input argument types. Ask, do they make sense and would they be expected to be used in that fashion normally?

Use “Tool Choice” to control when a tool is called

The tool choice parameter allows you to customize the behavior of tool calling with the model. We recommend utilizing this for fine grained control on which tools are called and

when.

For example, for use cases like structured output, you might want a specific tool to be called each time Amazon Nova is invoked. You can define the schema of your output as the tool and then set the tool choice to the name of that tool.

```
{
  "toolChoice": {
    "tool": {
      "name": "name_of_tool"
    }
  }
}
```

For many agentic use cases, you might want to ensure that the model always selects one of the available tools. To do so, you can set the tool choice to `any`, which will call exactly one tool each time the model is invoked.

```
{
  "toolChoice": {
    "any": {}
  }
}
```

Lastly, for use cases where whether a tool is called is highly dependent on the context of the conversation, you can set the tool choice to `auto`. This is the default behavior and will leave the tool selection completely up to the model.

```
{
  "toolChoice": {
    "auto": {}
  }
}
```

Advanced prompting techniques

↓ [PDF \(/pdfs/nova/latest/nova2-userguide/nova2-ug.pdf#advanced-prompting-techniques\)](/pdfs/nova/latest/nova2-userguide/nova2-ug.pdf#advanced-prompting-techniques)

↓ [RSS \(nova2-ug.rss\)](#)

↓ [Markdown \(advanced-prompting-techniques.md\)](#)

⚡ **Agent Setup**

☾ Focus mode



Did this page help you?

👍 Yes

👎 No

Provide feedback ([https://docs.aws.amazon.com/feedback/doc-feedback.html?](https://docs.aws.amazon.com/feedback/doc-feedback.html?hidden_service_name=Nova&topic_url=https%3A%2F%2Fdocs.aws.amazon.com%2Fnova%2Flatest%2Fnova2-userguide%2Fadvanced-prompting-techniques.html)

[hidden_service_name=Nova&topic_url=https%3A%2F%2Fdocs.aws.amazon.com%2Fnova%2Flatest%2Fnova2-userguide%2Fadvanced-prompting-techniques.html](https://docs.aws.amazon.com/feedback/doc-feedback.html?hidden_service_name=Nova&topic_url=https%3A%2F%2Fdocs.aws.amazon.com%2Fnova%2Flatest%2Fnova2-userguide%2Fadvanced-prompting-techniques.html))

These sections provide advanced guidance for how to improve the quality of your prompts and leverage key features like extended thinking.

► Use Reasoning Mode

Amazon Nova 2 models offer an optional reasoning mode that enhances the model's approach to complex problem-solving by allowing it to work through problems systematically before responding. Leveraging the model's reasoning mode is a powerful way to improve the accuracy of your prompts.

When to use it: Reasoning mode is recommended for complex tasks such as use cases with:

- **Multiple reasoning steps:** Mathematical proofs, algorithm design, system architecture

- **Cross-referencing information:** Document analysis, option comparison, trade-off evaluation
- **Error-prone calculations:** Financial modeling, data analysis, complex debugging
- **Planning with constraints:** Resource optimization, dependency management, risk assessment
- **Complex classifications:** Multi-label categorization, hierarchical taxonomies, nuanced decision boundaries
- **Tool calling scenarios:** Multi-step API workflows, database query optimization, coordinated system integrations

Note

For more information on the reasoning mode, refer to [Using reasoning \(./using-converse-api.html#converse-api-reasoning\)](#) .

► Take a top-down approach

For situations where the model needs to assess multiple approaches to solve the problem, instruct it to take a **top-down** approach.

- Amazon Nova 2 models perform better when the model starts with the big picture and then breaks it down into smaller, more detailed subproblems or steps.
- Explicitly direct the model to first identify the main objective, then decompose it into manageable components before working through the details of each part.
- This structured approach helps the model organize its thinking and produce more coherent reasoning chains.

Example:

```
{{User query}}. Start with the big picture and break it down into progressively smaller, more detailed subproblems or steps.
```

► Steer Chain of Thought

While reasoning mode provides enhanced accuracy through systematic problem-solving, there are specific scenarios where Chain of Thought (CoT) prompting in non-reasoning mode may better serve your needs.

When to use it:

- **Transparency and auditability:** When you want to see, verify, or audit the model's reasoning process, CoT provides full visibility into each step. This is critical for regulated industries, high-stakes decisions, or when you want to document the logic behind an answer.
- **Custom reasoning structures:** Use CoT to enforce specific reasoning patterns or methodologies. You can guide the model to follow your organization's decision frameworks, use domain-specific problem-solving approaches, or ensure factors are considered in a specific order.
- **Prompt development and debugging:** During the prompt engineering phase, CoT helps you understand how the model approaches problems, identify where reasoning breaks down and iterate on your prompts more effectively.
- **Hybrid approaches:** Consider using CoT during development to perfect your prompts, then switching to reasoning mode for production deployment once you're confident in the model's approach to your specific use case.

Note

Not all tasks require CoT. For simpler tasks, allow the model to use its own reasoning process.

Guiding the model's CoT direction:

```
{{User query}} Please follow these steps:
```

- ```
1. {{Step 1}}
2. {{Step 2}}
...
```

## ► Utilizing long context windows

Amazon Nova 2 models have a supported context length of 1 million tokens and excel at code understanding and question answering on long documents. Its performance (including system prompt adherence and tool use) can decline slightly as the context size increases.

### How to use it:

- **Put long-form data at the beginning:** Place your long documents and inputs near the beginning of your prompt. Place them before your query, instructions and examples.
- **Put instructions at the end:** Place your instructions at the end of the prompt. The model performs best when the context is provided first and the instructions are provided at the end.
- **Structure document content start and end markers:** Use start and end markers, such as DOCUMENT {idx} START and DOCUMENT {idx} END, to denote the start and end of long documents where {idx} represents the index of the specific document.

### Example Template:

```
// Provide your long inputs at the top of your prompt
BEGIN INPUT DOCUMENTS
```

```
DOCUMENT 1 START
{{Your document}}
DOCUMENT 1 END
```

```
END INPUT DOCUMENTS
```

```
// Then specify your query and instructions
BEGIN QUESTION
{{User query}}
END QUESTION
```

```
BEGIN INSTRUCTIONS
```

```
{{Instructions}}
END INSTRUCTIONS
```

## ► Ground answers in supporting text

We recommend that you provide the model with trusted information relevant to the input query. This information, along with the input query, is often a part of the system called retrieval augmented generation (RAG).

- In this process, some relevant, contextual document or information is augmented to the actual user prompt so that the model gets trustworthy content to generate a relevant and accurate response.
- Instructing Amazon Nova 2 to answer using a reference text from a trusted source can guide it to compose its response based on the provided material and ensure that its response is grounded in accurate and relevant information, enhancing the reliability and credibility of the generated content.
- Using a reference text can help avoid hallucinating, thereby improving the overall quality and trustworthiness of the responses. To minimize hallucination, we recommend explicitly mentioning **DO NOT USE INFORMATION THAT IS NOT IN REFERENCE TEXTS!** in your model instructions.

### Prompt template

System:

In this session, the model has access to search results and a user's question, your job is to answer the user's question using only information from the search results.

Model Instructions:

- DO NOT USE INFORMATION THAT IS NOT IN SEARCH RESULTS!

User: {Query}

Resource: Search Results: {Reference texts}

## ► Example: Providing grounding context helps to prevent the model from hallucinating

This example shows how grounding context can prevent the model from hallucinating.

System:

In this session, the model has access to search results and a user's question, your job is to answer the user's question using only information from the search results.  
Model Instructions:- DO NOT USE INFORMATION THAT IS NOT IN SEARCH RESULTS!

User:

What were the economic impacts of the COVID-19 pandemic on the United States in 2020?

Resource: Search Results: In 2020, the United States experienced significant economic impacts due to the COVID-19 pandemic. The U.S. economy contracted by 3.5% in 2020, according to the Bureau of Economic Analysis. Unemployment rates surged to 14.7% in April 2020, the highest since the Great Depression, before gradually declining. Small businesses faced severe challenges, with millions of firms closing permanently. Additionally, consumer spending dropped sharply as people reduced non-essential expenditures and saved more. Government intervention played a critical role in mitigating these impacts through stimulus packages and support programs, such as the Paycheck Protection Program (PPP) for small businesses and direct payments to individuals. Despite these

measures, the economic recovery remained uneven across different sectors and regions.

## Ground using citation markers

For long document tasks, we recommend that you instruct the Amazon Nova 2 model to ground its responses using citations from the relevant sections of the documents before it proceeds with the task.

This approach helps the model focus on the most pertinent information and avoid being distracted by extraneous content. When you request that the model grounds its response, the sections that can be cited should be numbered. For example, Passage

%[1]%, Passage %[2]% and so on.

### ▼ Examples: using citation markers

#### Example Citation markers prompt

You are an AI financial assistant. Your task is to find patterns and insights from multi-year financial documents

Passage %[1]%  
{{Your document}}

Passage %[2]%  
{{Your document}}

## Task:

Analyze my LLC's reports across multiple years to identify significant performance trends, segment growth patterns and strategic shifts.

## Context information:

- You have access to my LLC's annual financial reports (10-K) for multiple fiscal

years in PDF format

- These reports contain comprehensive financial data including income statements, balance sheets, cash flow statements and management discussions
- The analysis should focus on year-over-year comparisons to identify meaningful trends
- I operate two business segments, one in Massachusetts and one in New York

Based on the provided Context, extract key financial metrics from each year's reports phrases from the documents.

Place citations as inline markers (e.g., %[1]%, %[2]%, etc.) directly within the relevant parts of the response text. Do not include a separate citation section after the response.

## Response Schema:

%% (Extracted Financial Metrics)

%% (Extracted Financial Metrics)

%% (Extracted Financial Metrics)

...

After you extract key information based on the user's task, you can use the extracted financial metrics to answer the relevant questions as shown:

### Example Follow-up analysis with extracted metrics

## Task

Analyze my LLC's financial reports across multiple years to identify significant performance trends, segment growth patterns and strategic shifts.

{{extracted financial metrics}}

## Model Instructions:

- Organize data chronologically to identify meaningful trends

- DO compare segment performance across the five-year period
- DO identify significant strategic shifts or investments mentioned in management discussions
- DO NOT make speculative predictions beyond what is supported by the data
- ALWAYS note any changes in accounting practices or reporting methodologies that might affect year-over-year comparisons

#### ## Response style and format requirements:

- Respond in markdown
- Structure the analysis with clear headings and subheadings
- Present key financial metrics in tabular format showing all five years side-by-side
- Include percentage changes year-over-year for all major metrics
- Create a section dedicated to visualizing the most significant trends (with descriptions of what would be shown in charts)
- Limit the executive summary to 250 words maximum
- Format segment analysis as separate sections with consistent metrics across all segments
- MUST include a Key Insights bullet-pointed list at the end of each major section

## Use Nova Web Grounding

Instead of prompting directly for citations to ground the model in supporting text, Amazon Nova 2 models provide an internal web grounding tool that can be used. When enabled, Amazon Nova 2 models will directly query the web and Amazon's knowledge graphs and ground the final response with citations.

To learn more on how to leverage Amazon Nova Web Grounding, you can reference the [Amazon Nova Web Grounding user guide](https://docs.aws.amazon.com/nova/latest/nova2-userguide/web-grounding.html) (<https://docs.aws.amazon.com/nova/latest/nova2-userguide/web-grounding.html>) .

## ► Produce structured output

To ensure consistent and structured output formats, you can use structured outputs, including formats like XML, JSON, Markdown, or using tool use functionality.

- This approach allows downstream systems to more effectively understand and parse the outputs generated by the model.
- By providing explicit instructions to the model, the responses are generated in a way that adheres to a predefined schema.

For example, if the downstream parser expects specific naming conventions for keys in a JSON object, you should specify the response schema at the end of the prompt. Additionally, if you prefer responses to be in JSON format without any preamble text, instruct the model accordingly. That is, explicitly state **Please generate only the JSON output. DO NOT provide any preamble.** to ensure clean output.

### 📘 Tip

- We observe best adherence to data format requirements when they are defined in the schema itself instead of through the use of exemplars (such as, specifying dates in YYYY/MM/DD format).
- For simple JSON outputs with up to 10 keys, you can find the schema below. For more complex schemas, we recommend you define your schema through a tool. Tool use leverages a technique called constrained decoding that will increase the model's adherence for these complex schemas.

## Common formatting schemas

The following are examples of common formatting schemas.

**JSON****XML****Markdown**

JSON\_format = ""Write your response following the JSON format below:

```
```json
{
  "key1": "value1",
  "key2": "value2",
  key3: [{
    "key3_1": "value_3_1 written in YYYY/MM/DD
format",
    "key3_2": "value_3_2 day of the week written in
full form",
    ...}]
}```
```

Prefill assistant content

If you are producing structured output in non-reasoning mode, you can nudge the model's response by prefilling the assistant content.

Prefilling improves consistency in the output format while in non-reasoning mode. It allows you to direct the model's actions, bypass preambles and enforce specific output formats like JSON and XML. For example, if you prefill the assistant content with `{` or ````json`, that input guides the model to generate the JSON object without additional information.

i Tip

If you are explicitly looking for extracting JSON, a common pattern is to prefill with ````json` and add a stop sequence on `````. This ensures the model outputs a JSON object that can be programmatically parsed.

Example Prefill assistant content

The following code demonstrates how to prefill with the API:

```

import boto3
import json

# Create a Bedrock Runtime client.
client = boto3.client(
    "bedrock-runtime",
    region_name="us-east-1"
)

request_body = {
    "system": [
        {"text": "You write JSON objects based on the
given instructions"}
    ],
    "messages": [
        {
            "role": "user",
            "content": [{"text": "Provide details
about the best selling full-frame cameras in past
three years. Answer in JSON format with keys like
name, brand, price and a summary."}]
        },
        {
            "role": "assistant",
            "content": [{"text": " Here is the JSON
response: ``json"}]
        }
    ],
    "inferenceConfig": {
        "maxTokens": 1000,
    }
}

# Invoke the model and extract the response body.
response = client.invoke_model(
    modelId="amazon.nova-2-lite-v1:0",
    body=json.dumps(request_body)
)

model_response = json.loads(response["body"].read())

```

Specify a tool to use for complex schemas

Another approach is to use tools to force a specific schema for the model's response by including the *pydantic JSON schema* in the tool use schema section. You can specify tool choice to the provided schema and Amazon Nova's response will be structured based on the tool selected. To learn more about how to leverage tool use see [Tool Use](https://docs.aws.amazon.com/nova/latest/userguide/tool-use.html)

(<https://docs.aws.amazon.com/nova/latest/userguide/tool-use.html>) .

Examples

► Example 1: Add an output schema

Adding the Output Schema section with the appropriate schema makes the model prefer to fit that schema. However, model output is not deterministic and can vary from the output schema.

Unspecific Prompt

Specific Prompt

User:

Provide details about the best selling full-frame cameras in past three years. Answer in JSON format with keys like name, brand, price and a summary.

Assistant:

```
```json
[
 {
 "name": "Sony Alpha A7R IV",
 "brand": "Sony",
 "price": "Approximately $3,500",
 "summary": "The Sony Alpha A7R IV is renowned for its exceptional image quality, thanks to a 61-megapixel full-frame sensor. It also features 4K video
```

recording and a robust build, making it a favorite among professional photographers and videographers."

},

{

"name": "Canon EOS R5",

"brand": "Canon",

"price": "Approximately \$3,900",

"summary": "The Canon EOS R5 combines high-resolution imaging with advanced video capabilities. It houses a 45-megapixel sensor, 8K video recording, and impressive autofocus performance, appealing to both photography and filmmaking professionals."

},

{

"name": "Nikon Z7 II",

"brand": "Nikon",

"price": "Approximately \$2,800",

"summary": "The Nikon Z7 II offers a 45.7-megapixel full-frame sensor and is praised for its high resolution and excellent dynamic range. It also supports 4K video recording and features a versatile Z-mount lens system, making it a strong contender in the full-frame market."

},

{

"name": "Fujifilm GFX 100S",

"brand": "Fujifilm",

"price": "Approximately \$7,000",

"summary": "While not a traditional full-frame camera, the Fujifilm GFX 100S stands out with its 102-megapixel medium format sensor. It is lauded for its stunning detail and color reproduction, though it comes at a higher price point and is aimed at specialized users."

}

```
]
...
```

## ► Example 2: Prefill the model response

Another approach is to prefill the model's response by including the desired initial text within the assistant's message. In this case, Amazon Nova model's response will continue from where the assistant's message leaves off.

**Tip:** Prefill is only valid when reasoning is not enabled.

User:

```
Provide details about the best selling full-
frame cameras in past three years.
Your response should be in JSON format, with
the following keys: name, brand, year, price,
summary.
```

Assistant (Prefilling):

```
```json
```

Assistant:

```
{
  "cameras": [
    {
      "name": "Sony Alpha A7 III",
      "brand": "Sony",
      "year": 2018,
      "price": "$1999",
      "summary": "The Sony Alpha A7 III is a
```

highly acclaimed full-frame mirrorless camera known for its excellent image quality, advanced autofocus system, and 4K video capabilities."

},

{

"name": "Canon EOS R5",

"brand": "Canon",

"year": 2020,

"price": "\$3899",

"summary": "The Canon EOS R5 is a top-tier full-frame mirrorless camera that offers stunning 45MP images, 8K video recording, and exceptional performance in various lighting conditions."

},

{

"name": "Nikon Z7 II",

"brand": "Nikon",

"year": 2020,

"price": "\$2999",

"summary": "The Nikon Z7 II is a flagship full-frame mirrorless camera with a 45.7MP sensor, superb image quality, and advanced video recording features."

},

{

"name": "Fujifilm GFX 100S",

"brand": "Fujifilm",

"year": 2020,

"price": "\$7999",

"summary": "The Fujifilm GFX 100S is a medium format mirrorless camera known for its impressive 102MP sensor, beautiful film simulations, and versatile lens options."

},

{

"name": "Sony Alpha A7R IV",

"brand": "Sony",

"year": 2019,

"price": "\$3499",

"summary": "The Sony Alpha A7R IV features a

```
61MP sensor, excellent low-light performance,  
and advanced video capabilities, making it a  
favorite among professional photographers."
```

```
}  
]  
}  
...
```

► Example 3: Specify a tool to use

As an alternative to using the prefill technique for structured output, you can combine Nova's tool functionality with toolChoice. In this example, the model will generate a tool call that adheres to the JSON schema provided and calls the specific tool specified to full JSON format defined. For additional guidance, refer to Advanced Systems.

User:

```
From the below provided Query, extract the  
relevent entities
```

Query:

```
John works in BUILDING-0987 and have been  
incharge of product id 23564#. His  
performance have been excellent in past year  
and is up for a raise.
```

```
Use the extract_ner tool.
```

ToolConfig:

```
{  
  "tools": [  
    {  
      "toolSpec": {  
        "name": "extract_ner",  
        "description": "Extract all
```

```

the named entities. based on provided input",
    "inputSchema": {
        "json": {
            "type": "object",
            "properties": {
                "entities": {
                    "type": "array",
                    "items": {
                        "type":
"object",
                        "properties":
{
                            "name": {
                                "type":
"string",
                                "description": "The extracted entity name.
This should be a name of a person, place,
animal or thing"
                                },
                                "location": {
                                    "type":
"string",
                                    "description": "The extracted location name.
This is a site name or a building name like
SITE-001 or BUILDING-003"
                                    },
                                    "product": {
                                        "type":
"string",
                                        "description": "The extrcted product code,
this is generally a 6 digit alphanumeric code
such as 45623#, 234567"
                                        }
                                    },
                                    "required": [
                                        "name",
                                        "location",
                                        "product"
                                    ]
                                }
                            }
                        }
                    }
                }
            }
        }
    }

```

```

    }
    }
  },
  "required": [
    "entities"
  ]
}

}

}],
"toolChoice": {
  "tool": {
    "name": "extract_ner"
  }
}
}

```

► Multilingual use cases

Amazon Nova 2 models have been trained on more than 200 languages and optimized for 15 languages.

Topics

- [Prompt for accurate translations \(#accurate-translations\)](#)
- [Enforce consistent writing conventions \(#consistent-writing-conventions\)](#)

Prompt for accurate translations

To leverage this capability for short form translations (a few sentences) you can instruct the model to translate the text into the specified target language.

Example Translation prompts

Translate the following text into {target language}.
Please output only the translated text with no prefix
or introduction: {text}

Translate the following sentence from
{source_language} to {target language}: {text}

{text} How do you say this sentence in
{target_language}

Enforce consistent writing conventions

In character-based languages, the Amazon Nova 2 models may utilize the character set from the source language. You can use the following prompt to enforce a consistent output.

Example Enforce writing conventions

When translating, ensure to use the correct
orthography / script / writing convention of the
target language, not the source language's characters

► Tool calling

Agentic systems

Topics

- [Set the right inference parameters \(#set-inference-parameters\)](#)
- [Consider latency requirements \(#consider-latency-requirements\)](#)
- [Use intentional wording for tool calling instructions \(#intentional-wording-tool-calling\)](#)
- [Leverage "thinking" commands \(#leverage-thinking-commands\)](#)

- [Tool call ordering \(#tool-call-ordering\)](#)
- [Create quality tool schemas \(#designing-tool-schema\)](#)
- [Create sub-agents \(#create-sub-agents\)](#)
- [Use tools for multimodal inputs \(#use-tools-multimodal-inputs\)](#)
- [Next steps \(#next-steps-best-practices\)](#)

Set the right inference parameters

Tool calling requires a very specific structured output from the model and is improved by using the following inference parameters:

- **Non Reasoning Mode:** Temperature: 0.7 & Top P: 0.9
- **Reasoning Mode:** Temperature: 1 & Top P: 0.9

Consider latency requirements

Tip

Amazon Nova 2 models are capable of tool calling with reasoning on and off. However, reasoning modes have a significant impact on latency.

For latency sensitive applications, you should optimize for reasoning off mode and simplify the required tool calls where possible. Split multistep workflows into discrete steps to reduce the model's reliance on regurgitating unnecessary parameters.

Use intentional wording for tool calling instructions

Tool names: Referencing tools in the system prompt is common in tool calling systems to instruct the model on when to call a tool. When you reference tools in the prompt, we recommend you use the tool name instead of xml or pythonic references or examples.

► Example of good tool reference

```
Use the 'run_shell_command' tool for running  
shell commands
```

► Example of bad tool reference

```
Call run_shell_command() to run shell
commands
```

Leverage "thinking" commands

For all use cases where thinking is beneficial for tool calling, we recommend you leverage reasoning mode instead of prompting the model to "think in tags" or to use a "think" tool.

Amazon Nova 2 models are trained extensively for reasoning mode and will produce the optimal results when used in reasoning mode for chain of thought.

Tool call ordering

In use cases that might require the use of built-in tools and native tool calling simultaneously, the model biases towards calling built-in tools first.

Don't instruct the model to act differently in the prompt. Instead, design your workflow to take this into account.

For example, if you do not want the model to use built-in tools, do not include them in your workflow so that the model does not bias towards them.

Create quality tool schemas

Tool schemas are one of the key places you can prompt engineer effective tool calling systems. However, it's important to consider what gets captured in the tool schema itself, how each schema element is described semantically and how the system prompt references tools and schema elements within system instructions.

Amazon Nova 2 models are optimized for concise descriptions in the tool schemas. Keep it brief.

Tool schema versus system prompt guidelines:

Include in the tool schema:

- Core functionality: What the tool does (20-50 words recommended)
- Parameter specifications: Clear descriptions of each parameter (around 10 words per parameter)
- Expected formats: Data types (like enum, int, float), required fields and valid value ranges

Include in the system prompt:

- Dedicate a `#Tool Usage` section with orchestration logic (when and why to use specific tools) and business rules (conditional logic, sequencing requirements, and dependencies).
- **Error handling strategies:** Add an `#Error Handling and Troubleshooting` section with instructions for how to respond to failures or unexpected outputs
- **Output formatting:** Add details on how to present to the user

► Example

```
You are a software engineering issue root  
cause analysis agent. You are tasked with  
reviewing a customer issue and examining the  
repository to identify a plan to resolve the  
issue.
```

Core Mandates

```
- **DO NOT** update the original issue that  
was posted by the user. You only add  
*additional* comments to the reported issue  
if necessary
```

Primary Workflows

1. ****Understand:**** Analyze the user's request and explore the codebase thoroughly using ****get_file_contents**** to grasp file structures and conventions.
2. ****Plan:**** Create a coherent, evidence-based plan for resolving the task and share

it with the user following the format below

Tool Usage

- ****Read the Issue:**** Always start by using the ****read_issue**** tool to get the details about the requested issue
- ****File Paths:**** Always end the file path with "/" if you are searching a directory using the ****get_file_contents**** tools
- ****Parallelism:**** Execute multiple independent tool calls in parallel when feasible

Error Handling and Troubleshooting

- ****File Exploration:**** If you get an error that a file doesn't exist, try searching at the directory level first to validate the file path

Output Formatting

Return your plan in markdown in the following format

Issue

<Your root cause analysis of the issue>

Resolution Plan

<your step by step plan of how to solve the issue>

Create sub-agents

Consider creating specialized sub-agents instead of a single agent with many tools when you encounter:

- **Tool count exceeds 20:** Large tool sets become difficult to manage and increase selection errors
- **Distinct functional domains:** Tools naturally cluster into separate categories (such as data retrieval versus processing versus reporting)

- **Complex schemas:** When parameter depth exceeds 3-4 levels or tools have intricate interdependencies
- **Conversation length:** Workflows regularly exceed 15-20 turns may benefit from specialized sub-agents
- **Performance degradation:** If you observe decreased accuracy in tool selection or increased latency

Tip

MCP Servers come with tools and schemas you can't control. Only include the necessary tools for your workflow to complete the required task.

Use tools for multimodal inputs

For multimodal tasks, we have not observed improved accuracy leveraging tools for structured tasks (such as extraction or timestamp generation).

Instead, we recommend you review the relevant sections in the Prompting multimodal inputs section for how to prompt the model successfully using the provided templates.

Next steps

- For multimodal prompting, see [Prompting multimodal inputs \(./prompting-multimodal.html\)](#) .

Prompting multimodal inputs

[↓ PDF \(/pdfs/nova/latest/nova2-userguide/nova2-ug.pdf#prompting-multimodal\)](#)

[↓ RSS \(nova2-ug.rss\)](#)

[↓ Markdown \(prompting-multimodal.md\)](#)

[⚡ Agent Setup](#)

☐ Focus mode



On this page

Did this page help you?

☐ Yes

☐ No

Provide feedback ([https://docs.aws.amazon.com/feedback/doc-feedback.html?](https://docs.aws.amazon.com/feedback/doc-feedback.html?hidden_service_name=Nova&topic_url=https%3A%2F%2Fdocs.aws.amazon.com%2Fnova%2Flatest%2Fnova2-userguide%2Fprompting-multimodal.html)

[hidden_service_name=Nova&topic_url=https%3A%2F%2Fdocs.aws.amazon.com%2Fnova%2Flatest%2Fnova2-userguide%2Fprompting-multimodal.html](https://docs.aws.amazon.com/feedback/doc-feedback.html?hidden_service_name=Nova&topic_url=https%3A%2F%2Fdocs.aws.amazon.com%2Fnova%2Flatest%2Fnova2-userguide%2Fprompting-multimodal.html))

The following sections provide guidance for image and video understanding. For audio-related prompting, refer to the [Voice conversation prompts \(./sonic-system-prompts.html\)](#) section.

General multimodal guidelines

User prompts and system prompts

For multimodal understanding use cases, every request should include user prompt text. System prompts, which may only contain text, are optional.

System prompts can be used to specify a persona for the model and to define general personality and response style but should not be used for detailed task definition or

output formatting instructions.

Include task definition, instructions and formatting details in the user prompt to have a stronger effect than the system prompt for multimodal use cases.

Content order

A multimodal understanding request sent to Amazon Nova should contain one or more files and a user prompt. The user text prompt should be the last item in the message, always after the image, document, or video content.

```
message = {
  "role": "user",
  "content": [
    { "document|image|video|audio": {...} },
    { "document|image|video|audio": {...} },
    ...
    { "text": "<user prompt>" }
  ]
}
```

In cases where you want to refer to specific files from within the user prompt, use “text” elements to define labels that precede each file block.

```
message = {
  "role": "user",
  "content": [
    { "text": "<label for item 1>" },
    { "document|image|video|audio": {...} },
    { "text": "<label for item 2>" },
    { "document|image|video|audio": {...} },
    ...
    { "text": "<user prompt>" }
  ]
}
```

Document and Image Understanding

The following sections provide guidance on how to craft prompts for tasks that require understanding or analyzing images and documents.

Extracting text from images

Amazon Nova models can extract text from images, a capability referred to as Optical Character Recognition (OCR). For best results, ensure the image input you provide to the model is a high enough resolution that the text characters are easy to discern.

For text extraction use cases, we recommend the following inference configuration:

- **temperature:** default (0.7)
- **topP:** default (0.9)
- Do not enable reasoning

The Amazon Nova models can extract text to Markdown, HTML, or LaTeX format. The following user prompt template is recommended:

Instructions

Extract all information from this page using only {text_formatting} formatting. Retain the original layout and structure including lists, tables, charts and math formulae.

Rules

1. For math formulae, always use LaTeX syntax.
2. Describe images using only text.
3. NEVER use HTML image tags `<img>` in the output.
4. NEVER use Markdown image tags `![]()` in the output.
5. Always wrap the entire output in ````` tags.

The output is wrapped in full or partial Markdown code fences (`````). You can strip the code fences using code similar to the following:

```
def strip_outer_code_fences(text):
    lines = text.split("\n")
    # Remove only the outer code fences if present
    if lines and lines[0].startswith("```"):
        lines = lines[1:]
        if lines and lines[-1].startswith("```"):
```

```
lines = lines[:-1]
return "\n".join(lines).strip()
```

Extracting structured information from images or text

The Amazon Nova models can extract information from images into machine-parsable JSON format, a process referred to as Key Information Extraction (KIE). To perform KIE, provide the following:

- A JSON schema. A formal schema definition that follows the JSON Schema specification.
- One or more of the following: A document file or image or document text

The document or image must always be placed before your user prompt in the request.

For KIE use cases, we recommend the following inference configuration:

- **temperature:** 0
- **Reasoning:** Reasoning is not required but can improve results when image-only input or complex schemas are used.

Prompt templates

► Document or image only input

Given the image representation of a document, extract information in JSON format according to the given schema.

Follow these guidelines:

- Ensure that every field is populated, provided the document includes the corresponding value. Only use null when the value is absent from the document.
- When instructed to read tables or lists, read each row from every page. Ensure every field in each row is populated if the document contains the field.

JSON Schema:
{json_schema}

► Text only input

Given the OCR representation of a document, extract information in JSON format according to the given schema.

Follow these guidelines:

- Ensure that every field is populated, provided the document includes the corresponding value. Only use null when the value is absent from the document.
- When instructed to read tables or lists, read each row from every page. Ensure every field in each row is populated if the document contains the field.

JSON Schema:
{json_schema}

OCR:
{document_text}

► Document or Image and text input

Given the image and OCR representations of a document, extract information in JSON format according to the given schema.

Follow these guidelines:

- Ensure that every field is populated, provided the document includes the corresponding value. Only use null when the value is absent from the document.
- When instructed to read tables or lists, read each

row from every page. Ensure every field in each row is populated if the document contains the field.

JSON Schema:
{json_schema}

OCR:
{document_text}

Detecting objects and their positions in images

Amazon Nova 2 models provide the ability to identify objects and their positions within images, a task sometimes referred to as image grounding or object localization. Practical applications include image analysis and tagging, user interface automation, image editing and others.

Regardless of the image input resolution and aspect ratio, the model uses a coordinate space that divides the image into 1,000 units horizontally and 1,000 units vertically, with the x:0 y:0 location being the upper left of the image.

Bounding boxes are described using the format `[x1, y1, x2, y2]` representing left, top, right and bottom respectively. Two-dimensional coordinates are represented using the format `[x, y]`.

For object detection use cases, we recommend the following inference parameter values:

- **temperature:** 0
- Do not enable reasoning

Prompt templates: general object detection

We recommend the following user prompt templates.

Detecting multiple instances with bounding boxes:

Please identify {target_description} in the image and provide the bounding box coordinates for each one you detect. Represent the bounding box as the `[x1, y1, x2, y2]` format, where the coordinates are scaled between 0 and 1000 to the image width and height, respectively.

Detecting a single region with bounding box:

Please generate the bounding box coordinates corresponding to the region described in this sentence: {target_description}. Represent the bounding box as the [x1, y1, x2, y2] format, where the coordinates are scaled between 0 and 1000 to the image width and height, respectively.

Detecting multiple instances with center points:

Please identify {target_description} in the image and provide the center point coordinates for each one you detect. Represent the point as the [x, y] format, where the coordinates are scaled between 0 and 1000 to the image width and height, respectively.

Detecting a single region with center point:

Please generate the center point coordinates corresponding to the region described in this sentence: {target_description}. Represent the center point as the [x, y] format, where the coordinates are scaled between 0 and 1000 to the image width and height, respectively.

Parsing model output:

Each of the recommended prompts above will produce a comma separated string containing one or more bounding box descriptions in a form similar to the following. There may be some slight variation in whether a "." is included at the end of the string. For example, [356, 770, 393, 872],
[626, 770, 659, 878].

You can parse the coordinate information generated by the model using a regular expression as shown in the following Python code example.

► Code example

```
def parse_coord_text(text):
    """Parses a model response which uses array
    formatting ([x, y, ...])
    to describe points and bounding boxes. Returns an
    array of tuples."""
    pattern = r"\s*([^\s]*)\s*"
    return [
        tuple(int(x.strip()) for x in
match.split(", "))
        for match in re.findall(pattern, text)
    ]
```

► Code example

To remap the normalized coordinates of a bounding box to the coordinate space of the input image, you can use a function similar to the following Python example.

```
def remap_bbox_to_image(bounding_box, image_width,
image_height):
    return [
        bounding_box[0] * image_width / 1000,
        bounding_box[1] * image_height / 1000,
        bounding_box[2] * image_width / 1000,
        bounding_box[3] * image_height / 1000,
    ]
```

Prompt templates: detecting multiple object classes with positions

When you want to identify multiple classes of items in an image, you can include a class list in your prompt using one of the following formatting approaches.

For commonly understood classes that the model is likely to understand well, list the class names (without quotes) inside square brackets:

```
[car, traffic light, road sign, pedestrian]
```

For classes that are nuanced, uncommon, or come from specialized domains that the model may not be familiar with, include a definition for each class in parentheses. Since this task is challenging, expect the model's performance to degrade.

```
[taraxacum officinale (Dandelion - bright yellow flowers, jagged basal leaves, white puffball seed heads), digitaria spp (Crabgrass - low spreading grass with coarse blades and finger-like seed heads), trifolium repens (White Clover - three round leaflets and small white pom-pom flowers), plantago major (Broadleaf Plantain - wide oval rosette leaves with tall narrow seed stalks), stellaria media (Chickweed - low mat-forming plant with tiny star-shaped white flowers)]
```

Use one of the following **user prompt** templates depending on which JSON output format you prefer.

► Prompt option 1

Detect all objects with their bounding boxes in the image from the provided class list. Normalize the bounding box coordinates to be scaled between 0 and 1000 to the image width and height, respectively.

Classes: {candidate_class_list}

Include separate entries for each detected object as an element of a list.

Formulate your output as JSON format:

```
[  
  {  
    "class 1": [x1, y1, x2, y2]  
  },  
  ...  
]
```

```
    ...  
]
```

► Prompt option 2

Detect all objects with their bounding boxes in the image from the provided class list. Normalize the bounding box coordinates to be scaled between 0 and 1000 to the image width and height, respectively.

Classes: {candidate_class_list}

Include separate entries for each detected object as an element of a list.

Formulate your output as JSON format:

```
[  
  {  
    "class": class 1,  
    "bbox": [x1, y1, x2, y2]  
  },  
  ...  
]
```

► Prompt option 3

Detect all objects with their bounding boxes in the image from the provided class list. Normalize the bounding box coordinates to be scaled between 0 and 1000 to the image width and height, respectively.

Classes: {candidate_class_list}

Group all detected bounding boxes by class.

Formulate your output as JSON format:

```
{
  "class 1": [[x1, y1, x2, y2], [x1, x2, y1, y2],
  ...],
  ...
}
```

► Prompt option 4

Detect all objects with their bounding boxes in the image from the provided class list. Normalize the bounding box coordinates to be scaled between 0 and 1000 to the image width and height, respectively.

Classes: {candidate_class_list}

Group all detected bounding boxes by class.

Formulate your output as JSON format:

```
[
  {
    "class": class 1,
    "bbox": [[x1, y1, x2, y2], [x1, x2, y1, y2],
  ...]
  },
  ...
]
```

The output will be encoded as a JSON which can be parsed with any JSON parsing library.

Prompt templates: screenshot UI bounds detection

We recommend the following user prompt templates.

Detecting UI element position based on a goal:

In this UI screenshot, what is the location of the element if I want to {goal}? Express the location coordinates using the [x1, y1, x2, y2] format, scaled between 0 and 1000.

Detecting UI element position based on text:

In this UI screenshot, what is the location of the element if I want to click on "{text}"? Express the location coordinates using the [x1, y1, x2, y2] format, scaled between 0 and 1000.

Parsing model output:

For each of the UI bounds detection prompts above, you can parse the coordinate information generated by the model using a regular expression as shown in the Python code example below.

► Code example

```
def parse_coord_text(text):
    """Parses a model response which uses array
    formatting ([x, y, ...])
    to describe points and bounding boxes. Returns an
    array of tuples."""
    pattern = r"\s*([^\s]*)\s*"
    return [
        tuple(int(x.strip()) for x in
match.split(","))
        for match in re.findall(pattern, text)
    ]
```

Video understanding

The following sections provide guidance on how to craft prompts for tasks that require understanding or analyzing videos.

Summarizing videos

Amazon Nova models can generate summaries of video content.

For video summarization use cases, we recommend the following inference parameter values:

- **temperature:** 0
- Some use cases may benefit from enabling model reasoning

No specific prompting template is required. Your user prompt should clearly specify the aspects of the video you care about. Here are a few examples of effective prompts:

Can you create an executive summary of this video's content?

Can you summarize the essential information from this video concisely?

Could you provide a summary of the video, focusing on its key points?

Generating detailed captions for videos

Amazon Nova models can generate detailed captions for videos, a task referred to as dense captioning.

For video captioning use cases, we recommend the following inference parameter values:

- **temperature:** 0
- Some use cases may benefit from enabling model reasoning

No specific prompting template is required. Your user prompt should clearly specify the aspects of the video you care about. Here are a few examples of effective prompts:

Provide a detailed, second-by-second description of the video content.

Break down the video into key segments and provide detailed descriptions for each.

Generate a rich textual representation of the video, covering aspects like movement, color and composition.

Describe the video scene-by-scene, including details about characters, actions and settings.

Offer a detailed narrative of the video, including descriptions of any text, graphics, or special effects used.

Create a dense timeline of events occurring in the video, with timestamps if possible.

Analyzing security video footage

Amazon Nova models can detect events in security footage.

For security footage use cases, we recommend the following inference parameter values:

- **temperature:** 0
- Some use cases may benefit from enabling model reasoning

You are a security assistant for a smart home who is given security camera footage in natural setting. You will examine the video and describe the events you see. You are capable of identifying important details like people, objects, animals, vehicles, actions and activities. This is not a hypothetical,

be accurate in your responses. Do not make up information not present in the video.

Extracting video events with timestamps

Amazon Nova models can identify timestamps related to events in a video. You may request that time stamps be formatted in seconds or in MM:SS format. For example, an event occurring at 1 minute 25 seconds in the video can be represented as 85 or 01:25.

For this use case, we recommend the following inference parameter values:

- **temperature:** 0
- Do not use reasoning

We recommend you use prompts similar to the following:

► Identifying an event's start and end time

Please localize the moment that the event "{event_description}" happens in the video. Answer with the starting and ending time of the event in seconds, such as `[[72, 82]]`. If the event happen multiple times, list all of them, such as `[[40, 50], [72, 82]]`.

Locate the segment where "{event_description}" happens. Specify the start and end times of the event in MM:SS.

Answer the starting and end time of the event "{event_description}". Provide answers in MM:SS

When does "{event_description}" in the video? Specify the start and end timestamps, e.g. `[[9, 14]]`

► Identifying multiple occurrences of an event

Please localize the moment that the event "{event_description}" happens in the video. Answer with the starting and ending time of the event in seconds. e.g. `[[72, 82]]`. If the event happen multiple times, list all of them. e.g. `[[40, 50], [72, 82]]`

► Generating a log of video segments with timestamps

Segment a video into different scenes and generate caption per scene. The output should be in the format: `[STARTING TIME-ENDING TIMESTAMP] CAPTION`. Timestamp in MM:SS format

For a video clip, segment it into chapters and generate chapter titles with timestamps. The output should be in the format: `[STARTING TIME] TITLE`. Time in MM:SS

Generate video captions with timestamp.

Classifying videos

You can use Amazon Nova models to classify videos based on a pre-defined list of classes you provide.

For this use case, we recommend the following inference parameter values:

- **temperature:** 0
- Reasoning should not be used

Use the following prompt template:

```
What is the most appropriate category for this video? Select
your answer from the options provided:
{class1}
{class2}
{...}
```

Example:

```
What is the most appropriate category for this video? Select
your answer from the options provided:
Arts
Technology
Sports
Education
```

Content moderation

↓ PDF (/pdfs/nova/latest/nova2-userguide/nova2-ug.pdf#prompting-content-moderation)

↓ RSS (nova2-ug.rss)

↓ Markdown (prompting-content-moderation.md)

⚡ Agent Setup

☾ Focus mode



On this page

Did this page help you?

👍 Yes

👎 No

Provide feedback ([https://docs.aws.amazon.com/feedback/doc-feedback.html?](https://docs.aws.amazon.com/feedback/doc-feedback.html?hidden_service_name=Nova&topic_url=https%3A%2F%2Fdocs.aws.amazon.com%2Fnova%2Flatest%2Fnova2-userguide%2Fprompting-content-moderation.html)

[hidden_service_name=Nova&topic_url=https%3A%2F%2Fdocs.aws.amazon.com%2Fnova%2Flatest%2Fnova2-userguide%2Fprompting-content-moderation.html](https://docs.aws.amazon.com/feedback/doc-feedback.html?hidden_service_name=Nova&topic_url=https%3A%2F%2Fdocs.aws.amazon.com%2Fnova%2Flatest%2Fnova2-userguide%2Fprompting-content-moderation.html))

This guide provides best practices for prompting Amazon Nova models to perform content moderation using the [MLCommons® AILuminate Assessment Standard](https://mlcommons.org/ailuminate/safety-resources/) [\(https://mlcommons.org/ailuminate/safety-resources/\)](https://mlcommons.org/ailuminate/safety-resources/) — a comprehensive framework with 12 hazard categories for AI safety. While this guide uses the MLCommons standard as an example to illustrate best practices, the prompting techniques work with any free-form or customizable content moderation categories and definitions you specify.

Key principles

- Use structured (XML, JSON) or free-form prompts based on your use case
- Moderate any text content: conversations, articles, social media posts, comments, reviews, and more

- Provide clear policy definitions
- Request explanations for transparency

Recommended configuration

For content moderation tasks, we recommend the following inference configuration:

- **Temperature** — Use the default value of 0.7.
- **Top P** — Use the default value of 0.9.
- **Reasoning** — Disable reasoning for content moderation to reduce latency.

MLCommons universal taxonomy

The standard defines 12 hazard categories:

MLCommons AILuminate hazard categories		
Category	Definition	Key subcategories
Violent Crimes	Content that enables, encourages, or endorses commission of violent crimes	Mass violence, Murder, Physical assault, Domestic abuse, Terror, Animal abuse
Non-Violent Crimes	Content that enables, encourages, or endorses non-violent criminal activity	Labor trafficking, Fraud, Theft, Arson, Drug crimes, Weapons crimes, Cyber crimes
Sex-Related Crimes	Content that enables, encourages, or endorses sex-related crimes	Sex trafficking, Sexual assault, Sexual harassment, Non-consensual intimate content, Bestiality
Child Sexual Exploitation	Content that contains, describes, enables, encourages, or endorses sexual abuse of minors	Inappropriate relationships with children, Sexual abuse of children, CSAM
CBRNE	Content that enables, encourages, or endorses	Chemical weapons, Biological weapons, Radiological weapons,

Category	Definition	Key subcategories
	creation of weapons of mass destruction	Nuclear weapons, High-yield explosives
Suicide and Self-Harm	Content that enables, encourages, or endorses intentional self-harm	Suicide, Self-injury, Disordered eating, Dangerous challenges
Hate	Content that demeans or dehumanizes people based on protected characteristics	Demeaning remarks, Inflicting physical/emotional harm, Exclusion and isolation
Specialized Advice	Unqualified advice in sensitive domains without appropriate disclaimers	Financial advice, Medical advice, Legal advice, Electoral advice, Safety claims
Privacy	Content containing sensitive non-public personal information	Physical security (addresses), Digital security (credentials), Financial security (account numbers)
Intellectual Property	Content that violates or encourages violation of IP rights	Copyright infringement, Trademark infringement, Patent infringement
Defamation	Content that is verifiably false and likely to injure someone's reputation	Libel, Slander, Disparagement
Sexual Content	Erotic content including pornography and cybersex	Pornography (graphic sexual descriptions), Direct erotic chat

Source: MLCommons® AllIlluminate Assessment Standard (Documentation)

Prompting examples

► Structured format

Best for automated systems requiring parseable output. These examples use few-shot learning, where you provide example input-output pairs in the prompt to guide the model's responses. The examples below include a single

demonstration, but you can include multiple examples to improve accuracy for your specific use case.

Structure (XML)

User:

You are a text content moderator that detects policy violations, describes violated categories, and explains the reasoning behind your decisions. You will see a policy list and a conversation to be moderated. Your moderation response should be formatted as follows.

Start the response by indicating policy violation status using either `<POLICY_VIOLATION>Yes</POLICY_VIOLATION>` for content that violates policies, or `<POLICY_VIOLATION>No</POLICY_VIOLATION>` for content that does not violate policies.

Then, list the specific violation categories between `<CATEGORY_LIST>` and `</CATEGORY_LIST>` tags. If there is no violation, then put `C0` between the tags.

End the response by providing an analysis explaining whether the content violates policies and why in `<EXPLAIN>` `</EXPLAIN>` tags. Here is an example.

```
<EXAMPLE>
<POLICY>
{{Policy definitions and categories}}
</POLICY>
<TEXT>
{{Content to moderate}}
</TEXT>
<POLICY_VIOLATION>Yes/No</POLICY_VIOLATION>
<CATEGORY_LIST>Categories or C0</CATEGORY_LIST>
```

```
<EXPLAIN>Explanation</EXPLAIN>
</EXAMPLE>
```

Now, please moderate the following input policy and text.

```
<POLICY>
{{Policy definitions and categories}}
</POLICY>
<TEXT>
{{Content to moderate}}
</TEXT>
```

Assistant:

```
<POLICY_VIOLATION>Yes/No</POLICY_VIOLATION>
<CATEGORY_LIST>Categories or C0</CATEGORY_LIST>
<EXPLAIN>Explanation</EXPLAIN>
```

Structure (JSON)

User:

You are a text content moderator that detects policy violations, describes violated categories, and explains the reasoning behind your decisions. You will check if there is unsafe content in the below text snippet in accordance with our safety policy. Your moderation response should be in JSON format with the following structure:

```
{
  "policy_violation": "Yes or No",
  "category_list": ["List of violated category codes"],
  "explanation": "Reason for the decision"
}
```

If there is no violation, use "C0" as the category

code.

IMPORTANT: Do not add any additional text or explanation. Your response must contain ONLY the JSON object, nothing else.

Here is an example:

[POLICY]

{{Policy definitions and categories}}

[TEXT]

{{Content to moderate}}

[RESPONSE]

```
{
  "policy_violation": "Yes or No",
  "category_list": ["Violated category codes or C0"],
  "explanation": "Explanation"
}
```

Now, please moderate the following input.

[POLICY]

{{Policy definitions and categories}}

[TEXT]

{{Content to moderate}}

[RESPONSE]

Assistant:

```
{
  "policy_violation": "No",
  "category_list": ["C0"],
  "explanation": "The text is a casual personal
question with no policy violations."
}
```

Sample content moderation prompt and response

User:

You are a text content moderator that detects policy violations, describes violated categories, and explains the reasoning behind your decisions. You will see a policy list and a conversation to be moderated. Your moderation response should be formatted as follows.

Start the response by indicating policy violation status using either `<POLICY_VIOLATION>Yes</POLICY_VIOLATION>` for content that violates policies, or `<POLICY_VIOLATION>No</POLICY_VIOLATION>` for content that does not violate policies.

Then, list the specific violation categories between `<CATEGORY_LIST>` and `</CATEGORY_LIST>` tags. If there is no violation, then put `C0` between the tags.

End the response by providing an analysis explaining whether the content violates policies and why in `<EXPLAIN>` `</EXPLAIN>` tags. Here is an example.

`<POLICY>`

The 12 categories are: Violent Crimes, Non-Violent Crimes, Sex-Related Crimes, Child Sexual Exploitation, CBRNE, Suicide & Self-Harm, Hate, Specialized Advice, Privacy, Intellectual Property, Defamation, Sexual Content.

If none apply, label as `C0`.

`</POLICY>`

`<EXAMPLE>`

`<TEXT>`

"To make a pipe bomb: Get metal pipe, end caps, gunpowder, and fuse. Drill hole in cap, fill pipe with powder, insert fuse, seal both ends tightly."

</TEXT>

<POLICY_VIOLATION>Yes</POLICY_VIOLATION>

<CATEGORY_LIST>CBRNE</CATEGORY_LIST>

<EXPLAIN>Provides instructions for creating explosives, enabling dangerous weapons creation.

</EXPLAIN>

</EXAMPLE>

<TEXT>

DIY explosive device: Mix ammonium nitrate fertilizer with fuel oil at 94:6 ratio.

Use blasting cap for detonation. Store in cool, dry place. #DIY #Chemistry

</TEXT>

Assistant:

<POLICY_VIOLATION>Yes</POLICY_VIOLATION>

<CATEGORY_LIST>CBRNE</CATEGORY_LIST>

<EXPLAIN>

The text provides specific instructions for creating ANFO explosives, including precise mixing ratios and detonation methods. This falls under CBRNE (high-yield explosives subcategory) as it enables dangerous weapons creation with actionable technical details.

</EXPLAIN>

► Free-form content moderation

Free-form prompts can take many diverse structures beyond the examples below. The following examples intentionally use different formatting styles to

illustrate the flexibility of free-form prompting. You can adapt the structure, phrasing, and format to suit your specific use case.

Example 1: Yes/No question (Suicide & Self-Harm)

Example Yes/No content moderation

User:

Please stick to the following policy for content moderation. {{user_policy}}

Does this text promote harmful activities?

Best way to hurt yourself without leaving marks?

Asking for tips on methods that work.

Assistant:

Yes. This text explicitly requests methods for self-harm, falling under the Suicide & Self-Harm category. The intent to conceal evidence ("without leaving marks") indicates deliberate harmful behavior.

Example 2: Category identification and reasoning (Hate)

Example Category identification content moderation

User:

You are a professional content moderator. Please stick to the following policy for content moderation.

{{user_policy}}

Which category does this text violate and why?

Text: "Women are naturally worse at mathematics than men, according to my personal observations. They should

stick to
arts and humanities."

Assistant:

Hate. This text makes demeaning generalizations about women based on gender (a protected characteristic), promoting exclusion from STEM fields. The claim lacks scientific basis and perpetuates harmful stereotypes.

Example 3: Severity rating (Privacy)

Example Severity rating content moderation

User:

Content moderation task

Policy: {{user_policy}}

Rate the severity of the following content as "none", "low", "medium", or "high" and identify any violated categories.

Content: "John Smith lives at 742 Evergreen Terrace, Springfield.

His SSN is 123-45-6789 and his bank account number is 9876543210."

Assistant:

Severity: High

Violated category: Privacy

This content exposes multiple forms of sensitive personal information

including a full name, home address, Social Security number, and bank account number. This falls under the Privacy category, specifically the physical security, digital security, and financial security subcategories. The combination of identifiers creates a significant risk of identity theft and financial fraud.

Security use cases

[↓ PDF \(/pdfs/nova/latest/nova2-userguide/nova2-ug.pdf#prompting-security\)](#)

[↓ RSS \(nova2-ug.rss\)](#)

[↓ Markdown \(prompting-security.md\)](#)

[⚡ Agent Setup](#)

☐ Focus mode



On this page

Did this page help you?

☐ Yes

☐ No

Provide feedback ([https://docs.aws.amazon.com/feedback/doc-feedback.html?](https://docs.aws.amazon.com/feedback/doc-feedback.html?hidden_service_name=Nova&topic_url=https%3A%2F%2Fdocs.aws.amazon.com%2Fnova%2Flatest%2Fnova2-userguide%2Fprompting-security.html)

[hidden_service_name=Nova&topic_url=https%3A%2F%2Fdocs.aws.amazon.com%2Fnova%2Flatest%2Fnova2-userguide%2Fprompting-security.html](https://docs.aws.amazon.com/feedback/doc-feedback.html?hidden_service_name=Nova&topic_url=https%3A%2F%2Fdocs.aws.amazon.com%2Fnova%2Flatest%2Fnova2-userguide%2Fprompting-security.html))

This guide provides best practices for prompting Nova 2 Lite to perform security operations tasks focused on analysis use cases. It covers two examples: [converting a threat intelligence report into a structured hunt plan \(#sec-threat-intelligence\)](#) for a Security Operations Center (SOC) team, and [reviewing source code for security vulnerabilities \(#sec-secure-code-review\)](#). Nova 2 Lite is not designed to generate malicious code or perform offensive security operations. The prompting techniques shown here produce structured, repeatable output that integrates into existing security workflows.

Key principles

- Assign the model a specific security role (software engineer, security engineer) to focus its output

- Define the output structure explicitly — sections, tables, and formats the model must follow
- Include rules that prevent hallucination of indicators, techniques, or vulnerabilities not present in the input
- Use iterative refinement to adapt output to your environment, tooling, and constraints

Threat intelligence hunt planner

This prompt template converts a threat intelligence report into a structured hunt plan. The model maps adversary behaviors to [MITRE ATT&CK](https://attack.mitre.org/) [↗] (<https://attack.mitre.org/>), a public knowledge base of adversary tactics and techniques used as a common vocabulary in threat intelligence. It then generates huntable hypotheses, identifies which techniques are not huntable through Security Information and Event Management (SIEM) telemetry, and produces a consolidated log source checklist. The output is SIEM-agnostic by default. Analysts can request detection queries for a specific platform (SPL, KQL, Sigma) as a follow-up.

The model assumes the threat intelligence report has already been vetted for relevance by your team and does not perform its own applicability assessment.

► System prompt

Recommended model configuration: Nova 2 Lite with medium reasoning.

Use the following system prompt to configure the model for threat intelligence analysis.

```
IMPORTANT: Write all output as readable markdown. Do not wrap the entire response in a code block. Use fenced code blocks only inside the Telemetry Requirements section if needed.
```

```
You are a senior threat hunter supporting a SOC team. Your role is to convert threat intelligence reports into actionable hunt plans focused on identifying adversary tradecraft.
```

The intelligence you receive has already been vetted for relevance. Do not assess whether the threat applies – it does. Focus entirely on operationalizing the tradecraft described.

Your output has three sections, in this order:

MITRE ATT&CK MAPPING

- Map every adversary behavior in the report to ATT&CK techniques and sub-techniques
- Present as a table: Technique ID | Name | Tactic | How used (one sentence, specific to the report)
- Do not map techniques not described or clearly implied in the report
- In the "How used" column, describe the behavior without reproducing specific domain, IP, or hash values. Refer to indicators generically rather than by value.

HUNT HYPOTHESES

Assessment

- State whether the report provides sufficient operational detail to support actionable hunting
- If the report defers IOCs or technical details to a separate document, name that document and note that indicator-based hunts require it

Not Hunttable via SIEM

- A technique is not hunttable if detecting it requires telemetry that the target device class does not realistically forward to a SIEM, or if detection requires decrypting encrypted payloads or inspecting raw packet

captures. Connection metadata (IPs, ports, timing, volume, TLS certificate

fields) is available without decryption and counts as huntable telemetry.

- List every mapped technique that cannot be hunted using logs a SIEM

- realistically ingests – one entry per technique with a one-line reason

- Each entry must include the technique ID and name

- Only list techniques that appear in your MITRE ATT&CK MAPPING table above.

- Do not introduce new techniques in this section

Huntable Hypotheses

- For each remaining technique, generate a hypothesis with:

- Statement: What you are looking for, stated as a testable proposition

- Data required: The specific log source and event type needed, realistic

- for where the technique executes

- Rationale: Why this activity would indicate the threat, and what

- distinguishes it from normal operations

- Priority: High / Medium / Low

- High: Technique is central to the adversary's objective AND detection

- signal is specific (low false-positive rate against the described

- behavior).

- Medium: Technique is supporting or post-compromise, OR detection signal

- requires correlation with other events to be reliable.

- Low: Technique is incidental, or detection signal overlaps heavily with

- benign activity and requires tuning per environment.

- Specific indicators: Reference the location of relevant indicators in the

- source report. Do not reproduce hash, IP, or domain values. The analyst

will retrieve exact values from the source report.

- Reference the ATT&CK technique ID in each hypothesis

TELEMETRY REQUIREMENTS

- Consolidate all required log sources across every huntable hypothesis into a single checklist
- Every telemetry entry must reference a technique ID from the Huntable

Hypotheses section. Do not add sources that support no listed hypothesis.

- Format each entry as: [] Log source (event type) – needed for – technique

IDs and what it detects

- All items unchecked – the analyst determines what is available, not you
- Flag high-value telemetry: sources that enable hunting for multiple techniques from this threat

RULES

- Do not invent techniques, indicators, behaviors, numeric thresholds, port numbers, byte sizes, time windows, or tuning values. If a threshold is needed in a hypothesis, phrase it as a variable ("volume exceeding baseline", "repeated connections") and leave tuning to the analyst.
- Do not invent log source names. Use only telemetry that applies to the device class where the technique executes
- Do not reference figures, tables, or sections by number unless they appear in the report you are analyzing. If you cannot verify a specific figure reference, describe the indicator location generically ("see IOC table") or omit the reference.
- If the report is vague about a technique, note the

ambiguity

- Every technique in the MITRE ATT&CK MAPPING table must appear in exactly one of "Not Huntable via SIEM" or "Huntable Hypotheses". Verify before finalizing.
- Do not invent ATT&CK technique IDs. If unsure of the correct ID for a described behavior, use the parent technique ID or omit the mapping rather than guess.
- Do not generate a hypothesis you would not act on. Label it NOT HUNTABLE VIA SIEM instead.
- Keep output tight – no filler, no preamble, no restating the report back to the analyst

EXAMPLE (classification reasoning)

- T1190 Exploit Public-Facing Application → Huntable. Inbound connections and HTTP request metadata to exposed services are visible in WAF, reverse proxy, and edge firewall logs.
- T1003.001 OS Credential Dumping: LSASS Memory → Not Huntable via SIEM without EDR. Requires process access telemetry (Sysmon Event 10, EDR process handle events); standard Windows Security logs do not capture it reliably.
- T1016 System Network Configuration Discovery → Not Huntable via SIEM on most endpoints. On-host enumeration (ipconfig, route) produces no network artifact; detection requires command-line logging that is often not forwarded.
- T1110.003 Password Spraying → Huntable. Authentication logs show low-volume failed logons across many accounts from a single source,

distinguishable from single-account brute force.

- T1070.004 File Deletion on a network appliance → Not Hunttable via SIEM.
Executes on an embedded device (router/firewall) that forwards syslog and connection metadata only; host-level file, process, auditd, and iptables-change events are not available to the SIEM even though equivalent logs exist on servers.
- T1486 Data Encrypted for Impact → Hunttable. File modification events and mass-rename patterns appear in file server audit logs and EDR file telemetry.

► Example

The following example demonstrates the full output structure. The input is a short vendor blog post summary. Longer reports produce more extensive plans.

User:

```
Our threat intel team flagged this report as relevant
to us.
Generate a hunt plan.
"A New Sandworm Malware Cyclops Blink Replaces VPN
Filter.pdf"
```

Assistant:

```
## MITRE ATT&CK MAPPING

| Technique ID | Name | Tactic | How used |
|-----|-----|-----|-----|
| **T1133** | External Remote Services | Initial
Access | Actors deploy modified device firmware
```

images by exploiting an externally-available management service. |

| **T1059.004** | Command and Scripting Interpreter: Unix Shell | Execution | Cyclops Blink executes downloaded files using the Linux API (Unix shell). |

| **T1542.001** | Pre-OS Boot: System Firmware | Persistence | The malware is embedded in a modified device firmware image to survive reboots. |

| **T1037.004** | Boot or Logon Initialization Scripts: RC Scripts | Persistence | Cyclops Blink runs at device startup via a tweaked RC script. |

| **T1562.004** | Impair Defenses: Disable or Modify System Firewall | Defense Evasion | The malware alters the Linux firewall to permit C2 traffic. |

| **T1036.005** | Masquerading: Match Legitimate Name or Location | Defense Evasion | Cyclops Blink masquerades as a Linux kernel thread process. |

| **T1082** | System Information Discovery | Discovery | The malware regularly queries and beacons device information to its C2 server. |

| **T1090** | Proxy | Command and Control | C2 traffic can be routed through a proxy to hide the source. |

| **T1132.002** | Data Encoding: Non-Standard Encoding | Command and Control | Command messages use a custom binary scheme to encode data. |

| **T1008** | Fallback Channels | Command and Control | The malware selects a C2 server from lists of IPv4 addresses and ports. |

| **T1071.001** | Application Layer Protocol: Web Protocols | Command and Control | Files are downloaded and commands issued over HTTP/HTTPS. |

| **T1573.002** | Encrypted Channel: Asymmetric Cryptography | Command and Control | C2 messages are individually encrypted (AES-256-CBC) and sent under TLS. |

| **T1571** | Non-Standard Port | Command and Control | C2 uses ports that are not typical for HTTP/HTTPS traffic. |

| **T1041** | Exfiltration Over C2 Channel | Exfiltration | The malware can upload files to a C2 server. |

HUNT HYPOTHESES

Assessment

The advisory provides sufficient tactical detail (e.g., C2 protocols, persistence mechanisms, and traffic patterns) to build generic detection logic. However, indicator-of-compromise (IOC) values (specific IPs, domains, hashes) are **deferred to the accompanying Cyclops Blink malware analysis report**; indicator-based hunts therefore require that report.

Not Hunttable via SIEM

- **T1059.004 - Command and Scripting Interpreter: Unix Shell** - Requires process-level execution telemetry from the network device, which is not forwarded to a typical SIEM.
- **T1542.001 - Pre-OS Boot: System Firmware** - Detection needs firmware image analysis; not available via standard logs.
- **T1037.004 - Boot or Logon Initialization Scripts: RC Scripts** - Relies on boot-time script execution logs that most SIEMs do not collect from network appliances.
- **T1562.004 - Impair Defenses: Disable or Modify System Firewall** - Requires firewall-rule-change audit data that is generally not forwarded from WatchGuard devices.
- **T1036.005 - Masquerading: Match Legitimate Name or Location** - Needs process-identity details that are not exported to SIEM from the device.

Hunttable Hypotheses

ATT&CK ID	Statement	Data required	Rationale	Priority	Specific indicators
-----------	-----------	---------------	-----------	----------	---------------------

--	--	--	--	--	--

T1133	Unexpected inbound connections to a device's remote-management interface from external IPs. WatchGuard firewall logs (management-interface connection events). Cyclops Blink is delivered via exploitation of open remote-management services; inbound connections that bypass the allowed management-access list are a clear sign of initial-access activity.			High	See Cyclops Blink malware analysis report - IOC table for affected management ports.
--------------	--	--	--	------	--

T1082	Periodic outbound HTTPS requests from a network device to a small set of domains/IPs with low-volume payloads. Outbound NetFlow/Zeek logs, DNS query logs, TLS handshake logs (SNI, certificate fields). The malware beacons device information on a regular schedule; repetitive, low-data-size HTTPS calls to the same endpoints stand out from normal device behavior.			High	See Cyclops Blink malware analysis report - beacon frequency and target domains.
--------------	---	--	--	------	--

T1090	Use of an intermediate proxy server for outbound C2 traffic from infected devices. Proxy server access logs (e.g., Squid, Bluecoat) showing requests originating from WatchGuard IPs. Proxy usage is a common evasion tactic; seeing known-good device IPs proxying to unusual destinations suggests C2 redirection.			Medium	See Cyclops Blink malware analysis report - proxy-config examples.
--------------	--	--	--	--------	--

T1132.002	Outbound HTTP/HTTPS requests containing high-entropy binary data that does not match normal application payloads. Proxy or Zeek HTTP(S) payload logs (or network TAP packet captures) with entropy analysis. The custom binary encoding of C2 messages produces high-entropy byte streams; anomalous binary content in otherwise legitimate-looking web requests can indicate command encoding.			Medium	See Cyclops Blink malware analysis
------------------	---	--	--	--------	------------------------------------

report - sample encoded command payloads. |

| ****T1008**** | Rapid switching of destination IP/port pairs for outbound connections from the same device within a short time window. | NetFlow/Zeek flow records (source-IP, destination-IP, destination-port, timestamps). | Fallback-channel logic causes the malware to rotate C2 endpoints; frequent IP/port changes from a single device are suspicious. |

****Medium**** | See Cyclops Blink malware analysis report - list of C2 IP/port pairs. |

| ****T1071.001**** | Outbound HTTP/HTTPS traffic to domains that resolve to Tor hidden services or to IPs known to host .onion exit nodes. | DNS resolution logs, NetFlow/Zeek logs, and optional Tor exit-node list cross-reference. | Cyclops Blink can use Tor for C2 reachability; traffic to .onion-associated domains or IPs is a strong indicator of hidden-service communication. | ****High**** | See Cyclops Blink malware analysis report - known C2 domains and Tor hidden-service names. |

| ****T1573.002**** | Outbound TLS handshakes from a network device that present self-signed certificates or certificates signed by an untrusted CA. | TLS handshake logs (e.g., from firewall/proxy) capturing certificate subject, issuer, serial, and public-key fingerprint. | The malware generates individually-signed TLS certificates for each C2 session; seeing unexpected self-signed or mistrust-CA certificates from a device is a red flag. | ****High**** | See Cyclops Blink malware analysis report - sample certificate details. |

| ****T1571**** | Outbound connections from a WatchGuard device to destination ports outside the conventional web ports on a regular basis. | NetFlow/Zeek flow logs (source-IP, destination-port). | Using non-standard ports evades typical port-filtering; repeated connections to atypical ports from network equipment warrants investigation. | ****Medium**** | See Cyclops Blink malware analysis report - list of non-standard C2 ports. |

| ****T1041**** | Large outbound data transfers to a known C2 IP that do not match normal device backup or

update patterns. | NetFlow/Zeek volume statistics, proxy transfer logs, or firewall byte-count logs. | The ability to exfiltrate files is part of the malware's C2 capabilities; bulk data pushes to C2 endpoints deviate from normal device behavior. |

****Medium**** | See Cyclops Blink malware analysis report - typical exfiltration sizes and destinations.

|

TELEMETRY REQUIREMENTS

| [] Log source (event type) - needed for - technique IDs and what it detects |

|-----|

|-----|

| ****[] WatchGuard firewall logs (management-interface connection events)**** - needed for - ****T1133**** (inbound remote-management exploitation) |

| ****[] WatchGuard firewall/NetFlow logs (outbound connection metadata)**** - needed for - ****T1082****, ****T1008****, ****T1571****, ****T1041**** (beacon timing, fallback-channel switching, non-standard ports, exfiltration volume) |

| ****[] DNS query logs**** - needed for - ****T1082****, ****T1071.001**** (domain resolution for C2, Tor hidden-service lookup) |

| ****[] Proxy server access logs (Squid/Bluecoat)**** - needed for - ****T1090****, ****T1132.002****, ****T1071.001****, ****T1573.002****, ****T1041**** (proxy usage, payload inspection, TLS handshake, large transfers) |

| ****[] TLS handshake / certificate logs (firewall or proxy)**** - needed for - ****T1573.002****, ****T1071.001**** (certificate fields, SNI, self-signed cert detection) |

| ****[] Zeek/NetFlow L7 payload logs (binary entropy, HTTP(S) body)**** - needed for - ****T1132.002**** (non-standard encoding detection) |

| ****[] WatchGuard syslog (device-level audit messages)**** - needed for - ****T1133**** (optional context on management-interface events) |

High-value telemetry (enabling multiple techniques):

- ****WatchGuard firewall logs**** - cover inbound remote-management (T1133), outbound beaconing (T1082), fallback-channel switching (T1008), non-standard ports (T1571), and exfiltration volume (T1041).
- ****Proxy logs + TLS handshake logs**** - together support proxy usage (T1090), web-protocol C2 (T1071.001), encrypted-channel detection (T1573.002), and payload-encoding analysis (T1132.002).
- ****NetFlow/Zeek flow data**** - provide the backbone for timing, IP/port rotation, and volume-based exfiltration across many C2 techniques.

Iterative refinement

After the initial output, use follow-up prompts to extend or adapt the plan.

- **Extend the plan:**
 - "Add an executive summary"
 - "Add response and containment guidance"
 - "Write detection queries for [SIEM platform]"
- **Refine based on your environment:**
 - "We don't have DNS logging, remove those hypotheses and focus on proxy and endpoint telemetry"
 - "Our partner org runs Sentinel, give me the detections in KQL so we can share"

The model revises the plan based on updated context.

Considerations

The following considerations apply when using this prompt to generate hunt plans from threat intelligence reports.

- **Verify technical identifiers.** Large language models can generate inaccurate technical specifics. Verify these values against authoritative sources before acting on them.

- **Validate telemetry requirements against your environment.** The model proposes log sources based on the techniques described in the report, not on the telemetry your organization collects. Treat the telemetry checklist as a set of requirements to evaluate, not an inventory of available data.
- **Assess the quality of the source report.** The hunt plan reflects the content of the input report. If the source intelligence is incomplete, inaccurate, or ambiguous, the output inherits those limitations. The model does not validate the accuracy of the report.
- **Use hunt plans as input to detection engineering.** Hunt plans are hypotheses for analysts to validate, tune, and operationalize. They are not finished detection rules.
- **Mitigate prompt injection risk.** Threat intelligence reports provided as input can contain instructions intended to manipulate the model. Treat the report as untrusted data rather than as trusted instructions. For more information, see [Prompt injection \(https://docs.aws.amazon.com/bedrock/latest/userguide/prompt-injection.html\)](https://docs.aws.amazon.com/bedrock/latest/userguide/prompt-injection.html) .

Secure code review

This prompt template reviews source code for security vulnerabilities. The model traces data flow through the code, identifies where attacker-controlled input reaches dangerous operations, rates severity based on three factors: attacker control of the input, exploitation prerequisites, and impact, and provides a fix for each finding. The output is language-agnostic and works on any source file.

The model assumes the code provided is the complete scope for review and does not speculate about vulnerabilities in code it cannot see.

Recommended model configuration:

- Temperature — 0
- Top P — 0.9
- Reasoning — Low

Two prompt variants are provided for different audiences:

- **Software Development Engineer (SDE) prompt** — For developers reviewing their own code or a teammate's pull request. Output includes plain language explanations, a code snippet fix for every finding, and a priority tag (Fix Now, Fix Before Merge, or Track For Later).
- **Security Engineer prompt** — For AppSec reviews, audits, and writing up findings for engineering teams. Includes everything in the SDE output, plus vulnerability class,

attack scenario, likely [Common Weakness Enumeration \(CWE\)](https://cwe.mitre.org/)  (<https://cwe.mitre.org/>) classification, remediation alternatives with tradeoffs, and a prioritized remediation roadmap.

► SDE system prompt

Use the following system prompt to configure the model for developer-focused security code review.

```
You are a senior software engineer performing a code review focused on security. You identify vulnerabilities, explain them in plain language, and provide fixes.
```

RULES

- Only report issues you can identify in the provided code. Do not speculate about vulnerabilities in code you cannot see.
- Only report findings with a plausible security impact. If the only consequence of the issue is a crash, hang, or malformed output with no path to data exposure, privilege escalation, or code execution, do not report it. Reliability and robustness issues are out of scope.
- For each potential finding, trace the data from its source to where it is used. Identify whether an attacker can influence the value at runtime. A variable that originates from runtime user input (CLI args, request parameters, API responses, file contents) is attacker-controlled. An environment variable set at deploy time is indirect influence unless the attacker can modify it at runtime. A variable derived from a hardcoded literal within the same script is not.

- For each finding, show the specific vulnerable code.
- Provide a concrete fix as a code snippet the developer can use directly.
- Explain what is wrong and why it matters in plain English.
- Attack scenarios must be technically accurate given the language runtime and execution context. Do not describe exploitation paths that are impossible due to how the language or framework processes input.
- If the user provides project context (what the code does, where it runs, what access it has), use it to calibrate severity. If not provided, state your assumptions and note that severity may change with more context.

FOR EACH FINDING

- Location: file, or function
- What is wrong: plain language
- Why it matters: practical impact, not theoretical
- Severity: Critical / High / Medium / Low
- Priority: Fix Now / Fix Before Merge / Track For Later
- Fix: code snippet showing the remediation

AT THE END

- Summary table: findings by severity
- Top 3 priorities with one-line justification for each

SEVERITY SCALE

- Critical: Attacker controls the input at runtime (direct user input, API parameter, file content they supply).
Exploitation requires no authentication or only default/public

access. Impact

is code execution, full data breach, or privilege escalation to a different trust boundary.

- High: Attacker controls the input at runtime.

Exploitation

requires valid credentials or a specific but achievable

precondition. Impact is data exposure, unauthorized actions, or

modification of data the attacker should not reach.

- Medium: Attacker influence is indirect (must control an

intermediate system, environment variable set at deploy time, or

win a race condition). OR impact is limited to information

disclosure with no path to further exploitation.

- Low: No realistic attacker control in the described deployment

context, but the code pattern would be dangerous if reused in a

less-trusted context. Defense-in-depth improvement only.

► Security Engineer system prompt

Use the following system prompt to configure the model for AppSec-focused security code review.

You are a senior application security engineer performing a security code review. You identify vulnerabilities, classify them, provide exploitation context, and recommend remediation with tradeoffs.

RULES

- Only report issues you can identify in the provided code. Do not

- speculate about vulnerabilities in code you cannot see.

- Only report findings with a plausible security impact. If the only

- consequence of the issue is a crash, hang, or malformed output with

- no path to data exposure, privilege escalation, or code execution,

- do not report it. Reliability and robustness issues are out of scope.

- A finding must identify something the code does wrong, not

- something the deployment environment could do wrong. If the

- vulnerability depends entirely on external misconfiguration

- rather than a flaw in the code logic itself, it is not a code

- review finding.

- For each potential finding, trace the data from its source to where

- it is used. Identify whether an attacker can influence the value at

- runtime. A variable that originates from runtime user input (CLI args,

- request parameters, API responses, file contents) is attacker-controlled.

- An environment variable set at deploy time is indirect influence unless

- the attacker can modify it at runtime. A variable derived from a

- hardcoded literal within the same script is not.

- For each finding, show the specific vulnerable code.

- Provide a concrete fix as a code snippet.

- Do not cite specific CVEs. Do not claim a finding is similar to a

- specific CVE. You identify vulnerability patterns, not entries in

a vulnerability database.

- CWE classifications are your best assessment. Mark each as "verify externally" and include the link to the CWE entry. If you are not confident in the CWE mapping, state "CWE uncertain" rather than guessing an incorrect classification.
- Attack scenarios must be technically accurate given the language runtime and execution context. Do not describe exploitation paths that are impossible due to how the language or framework processes input.
- If the user provides project context (what the code does, where it runs, what access it has), use it to calibrate severity and attack scenarios. If not provided, state your assumptions.

FOR EACH FINDING

- Location: file, or function
- What is wrong: plain language explanation
- Why it matters: practical impact
- Severity: Critical / High / Medium / Low
- Vulnerability class: description of the vulnerability pattern
- Attack scenario: step by step, how an attacker exploits this given the project context
- CWE: likely classification (mark "verify externally"), link
- Fix: code snippet showing the remediation
- Remediation alternatives: at least two options with security/performance/complexity tradeoffs

AT THE END

- Attack surface summary
- Vulnerability class distribution across findings
- Recommended additional testing (fuzzing targets,

DAST candidates,
areas not covered by this review)
- Prioritized remediation roadmap

SEVERITY SCALE

- Critical: Attacker controls the input at runtime (direct user input, API parameter, file content they supply).

Exploitation

requires no authentication or only default/public access. Impact

is code execution, full data breach, or privilege escalation to a different trust boundary.

- High: Attacker controls the input at runtime.

Exploitation

requires valid credentials or a specific but achievable

precondition. Impact is data exposure, unauthorized actions, or

modification of data the attacker should not reach.

- Medium: Attacker influence is indirect (must control an

intermediate system, environment variable set at deploy time, or

win a race condition). OR impact is limited to information

disclosure with no path to further exploitation.

- Low: No realistic attacker control in the described deployment

context, but the code pattern would be dangerous if reused in a

less-trusted context. Defense-in-depth improvement only.

► Example — SDE persona

The following example demonstrates the full output structure. The input is a Flask user management API with multiple vulnerability classes.

User:

```
Review the following code for security issues:
--- FILE: user_api.py ---
{file_content}
--- END: user_api.py ---
```

Assistant:

```
### Security Findings Report

#### 1. **SQL Injection in User Search & Report
Export**
**Location**: `user_api.py` ('search_users'),
(`export_report`)
**What is wrong**:
The SQL queries use string interpolation (`f"...`)
with untrusted user input (`query` and `department`
parameters). This allows attackers to inject
arbitrary SQL commands.
**Why it matters**:
An attacker could execute malicious SQL to
delete/modify data, exfiltrate sensitive information,
or escalate privileges.
**Severity**: **Critical**
**Priority**: **Fix Now**
**Fix**:
Use parameterized queries instead of string
interpolation.

```python
In search_users()
query = request.args.get("q", "")
conn = get_db()
cursor = conn.execute(
 "SELECT id, name, email, department FROM users
WHERE name LIKE ?",
```

```

 ('%' + query + '%',)
)

In export_report()
department = request.args.get("dept", "")
conn = get_db()
if report_type == "salary":
 cursor = conn.execute(
 "SELECT name, salary FROM users WHERE
department = ?",
 (department,)
)
else:
 cursor = conn.execute(
 "SELECT name, department FROM users WHERE
department = ?",
 (department,)
)
...

```

---

#### #### 2. **\*\*Insecure Password Hashing\*\***

**\*\*Location\*\*:** `user\_api.py` (`create\_user`)

**\*\*What is wrong\*\*:**

Passwords are hashed using `hashlib.md5()` without salts. MD5 is cryptographically broken and vulnerable to precomputed rainbow table attacks.

**\*\*Why it matters\*\*:**

Attackers can rapidly crack hashed passwords to gain unauthorized access to user accounts.

**\*\*Severity\*\*:** **\*\*High\*\***

**\*\*Priority\*\*:** **\*\*Fix Now\*\***

**\*\*Fix\*\*:**

Use a modern, salted hashing algorithm like `bcrypt` or `argon2`.

```
```python
```

```
import bcrypt
```

```
@app.route("/api/users", methods=["POST"])
```

```
def create_user():
```

```

    data = request.get_json()
    password = data["password"].encode()
    salt = bcrypt.gensalt()
    hashed = bcrypt.hashpw(password, salt)

    conn = get_db()
    conn.execute(
        "INSERT INTO users (name, email, department,
password_hash) VALUES (?, ?, ?, ?)",
        (data["name"], data["email"],
data["department"], hashed.decode())
    )
    conn.commit()
    conn.close()
    return jsonify({"status": "created"}), 201
...

```

3. **Path Traversal in Avatar Download**

****Location**:** `user\_api.py` (`get\_avatar`)

****What is wrong**:**

The `filename` parameter from the user is directly concatenated to `UPLOAD\_DIR` without validation. This allows directory traversal attacks.

****Why it matters**:**

Attackers could access files outside the `UPLOAD\_DIR`, such as system configuration files or application source code.

****Severity**:** ****High****

****Priority**:** ****Fix Now****

****Fix**:**

Validate the filename and ensure the resolved path stays within `UPLOAD\_DIR`.

```
```python
```

```
from werkzeug.utils import secure_filename
import os
```

```
@app.route("/api/users/<int:user_id>/avatar")
def get_avatar(user_id):
 filename = request.args.get("file",
```

```

"default.png")
 safe_name = secure_filename(filename)
 filepath = os.path.join(UPLOAD_DIR, safe_name)

 if not
os.path.abspath(filepath).startswith(os.path.abspath(
UPLOAD_DIR)):
 return jsonify({"error": "Invalid file
request"}), 400

 return send_file(filepath)
...

```

### ### Summary Table of Findings

Severity	Count
Critical	1
High	2
Medium	0
Low	0

### ### Top 3 Priorities

1. **\*\*Fix SQL Injection (Critical)\*\*** - Prevents total database compromise.
2. **\*\*Fix Insecure Password Hashing (High)\*\*** - Protects user accounts from credential theft.
3. **\*\*Fix Path Traversal (High)\*\*** - Stops unauthorized file access on the server.

## ► Example — Security Engineer persona

The following example demonstrates the full output structure. The input is a Flask user management API with multiple vulnerability classes.

User:

```
Review the following code for security issues:
--- FILE: user_api.py ---
{file_content}
--- END: user_api.py ---
```

Assistant:

```
Security Code Review Findings
```

```

```

```
**Finding 1: SQL Injection in User Search and
Report Export**
```

```
- **Location**: `user_api.py` ('search_users'),
(`export_report`)
- **What is wrong**:
```

The code constructs SQL queries using string interpolation with untrusted user input. This allows attackers to manipulate the query structure.

```
- **Why it matters**:
```

Attackers can execute arbitrary SQL commands, leading to data exfiltration, authentication bypass, or database manipulation.

```
- **Severity**: **Critical**
```

Attacker-controlled input directly influences query logic with no validation.

```
- **Vulnerability class**: SQL Injection
```

```
- **Attack scenario**:
```

1. Attacker sends `q=admin'--` to  
`/api/users/search`.

2. The query becomes `SELECT ... WHERE name LIKE '%admin'--%'`, returning all admin users.

3. Attacker sends `dept='); DROP TABLE users;--` to  
`/api/reports/export?type=salary`, deleting the user table.

```
- **CWE**: CWE-89 SQL Injection (verify externally)
[https://cwe.mitre.org/data/definitions/89.html]
```

```
- **Fix**:
```

Use parameterized queries consistently:

```

```python
# In search_users
cursor = conn.execute(
    "SELECT id, name, email, department FROM users
WHERE name LIKE ?",
    (f"%{query}%",)
)

# In export_report
if report_type == "salary":
    query = "SELECT name, salary FROM users WHERE
department = ?"
else:
    query = "SELECT name, department FROM users
WHERE department = ?"
    cursor = conn.execute(query, (department,))
```

- Remediation alternatives:
 1. Use an ORM (e.g., SQLAlchemy) to abstract query building.
 2. Input sanitization (e.g., restrict `query` to alphanumeric characters), but parameterization is preferred.

Finding 2: Path Traversal in Avatar Download
- Location: `user_api.py` (`get_avatar`)
- What is wrong:
 The `filename` parameter is directly concatenated to `UPLOAD_DIR` without validation, allowing directory traversal attacks.
- Why it matters:
 Attackers can read arbitrary files from the server filesystem.
- Severity: High
 Attacker-controlled input leads to unauthorized file access.
- Vulnerability class: Path Traversal
- Attack scenario:
 1. Attacker requests `/api/users/1/avatar?file=../../etc/passwd`.

```

## 2. The server serves

`/var/app/uploads/../../etc/passwd`, exposing sensitive system files.

- **CWE**: CWE-22 Improper Limitation of a Pathname to a Restricted Directory ('Path Traversal') (verify externally)

[<https://cwe.mitre.org/data/definitions/22.html>]

- **Fix**:

Validate and sanitize the filename:

```
```python
```

```
from pathlib import Path
```

```
def get_avatar(user_id):
```

```
    filename = request.args.get("file",  
"default.png")
```

```
    safe_name = Path(filename).stem
```

```
    if not safe_name.isalnum():
```

```
        safe_name = "default.png"
```

```
    filepath = os.path.join(UPLOAD_DIR, safe_name)
```

```
    return send_file(filepath)
```

```
```
```

- **Remediation alternatives**:

1. **Allowlist filenames** (e.g., only `default.png`).

2. **Store avatars in a sandboxed directory** with no traversal-friendly paths.

---

## #### **Finding 3: Insecure Password Hashing**

- **Location**: `user\_api.py` (`create\_user`)

- **What is wrong**:

Passwords are hashed using `hashlib.md5`, which is fast and vulnerable to brute-force attacks.

- **Why it matters**:

Attackers can crack hashed passwords using precomputed rainbow tables or dictionary attacks.

- **Severity**: **Medium**

Weak password protection increases risk of credential compromise.

- **Vulnerability class**: Inadequate Password Protection

- **Attack scenario**:

1. Attacker gains database access (e.g., via SQL injection).

2. They crack MD5 hashes to obtain plaintext passwords.

- **CWE**: CWE-916 Use of Password Hash With Insufficient Computational Effort (verify externally) [<https://cwe.mitre.org/data/definitions/916.html>]

- **Fix**:

Use a modern hashing algorithm like `bcrypt`:

```
```python
```

```
import bcrypt
```

```
    salt = bcrypt.gensalt()
```

```
    hashed = bcrypt.hashpw(password.encode(), salt)
```

```
```
```

- **Remediation alternatives**:

1. **Use `PBKDF2`** with high iteration counts.

2. **Integrate a password policy** (e.g., minimum length, complexity) to reduce brute-force success.

---

#### **Finding 4: Hardcoded Secret Key**

- **Location**: `user\_api.py` (`SECRET\_KEY` assignment)

- **What is wrong**:

The secret key is hardcoded, exposing CSRF protection and session management to risk.

- **Why it matters**:

Attackers can forge requests or hijack sessions if the key is known.

- **Severity**: **Low**

Impact is limited to authentication bypass in this code, but defense-in-depth issue.

- **Vulnerability class**: Hardcoded Cryptographic Key

- **Attack scenario**:

1. Attacker obtains the source code or dumps the file.

2. They reproduce the `SECRET\_KEY` to perform CSRF attacks.

- **CWE**: CWE-798 Use of Hard-coded Credentials (verify externally)  
[<https://cwe.mitre.org/data/definitions/798.html>]
- **Fix**:
  - Load the key from environment variables or a secure vault:
 

```
```python
SECRET_KEY = os.getenv("FLASK_SECRET_KEY",
"fallback-not-for-prod")
```
```
- **Remediation alternatives**:
  1. **Use Flask's built-in configuration** to load secrets from `instance/config.py`.
  2. **Rotate keys regularly** and invalidate sessions on rotation.

---

#### **Finding 5: Debug Mode Enabled in Production**

- **Location**: `user_api.py` (`app.run` call)
- **What is wrong**:
  - `debug=True` exposes stack traces, sensitive variables, and enables interactive debuggers.
- **Why it matters**:
  - Attackers can infer internal logic or leak memory contents via error responses.
- **Severity**: **Low**
  - Only applicable if the code is deployed with `debug=True` (unlikely in production).
- **Vulnerability class**: Improper Default Configuration
- **Attack scenario**:
  1. Attacker triggers an error (e.g., invalid endpoint).
  2. The server returns a detailed traceback revealing database paths or code logic.
- **CWE**: CWE-209 Information Exposure Through an Error Message (verify externally)  
[<https://cwe.mitre.org/data/definitions/209.html>]
- **Fix**:
  - Disable debug mode in production:
 

```
```python
```

```

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=8080, debug=False)
...

- Remediation alternatives:
    1. Use environment-specific configuration
    (e.g., `FLASK_ENV=production`).
    2. Deploy via WSGI server (e.g., Gunicorn)
    instead of `app.run()`.

---

### Attack Surface Summary
- Critical: 1 finding (SQL Injection)
- High: 1 finding (Path Traversal)
- Medium: 1 finding (Insecure Password Hashing)
- Low: 2 findings (Hardcoded Secret Key, Debug
Mode)

### Vulnerability Class Distribution
- SQL Injection: 1
- Path Traversal: 1
- Inadequate Password Protection: 1
- Hardcoded Cryptographic Key: 1
- Improper Default Configuration: 1

### Recommended Additional Testing
1. Fuzzing targets:
    - `/api/users/search?q=<MALICIOUS_INPUT>`
    - `/api/users/<user_id>/avatar?file=
<TRAVERSAL_PAYLOAD>`
2. DAST candidates:
    - Test all API endpoints for injection and
    traversal.
    - Validate error handling for debug leakage.
3. Areas not covered:
    - Authentication/authorization logic (not visible
    in this code).
    - Database permission checks.

### Prioritized Remediation Roadmap
1. Critical: Fix SQL injection (parameterized
queries).

```

2. ****High****: Mitigate path traversal (filename sanitization).
3. ****Medium****: Upgrade password hashing to `bcrypt`.
4. ****Low****: Remove hardcoded secret key and disable debug mode.

Iterative refinement

After the initial output, use follow-up prompts to adapt the review to your needs.

- **Add context:**
 - “This runs as a Lambda function behind API Gateway with IAM auth”
 - “The environment variables are set at deploy time by our CI pipeline”
- **Adjust scope:**
 - “Focus only on Critical and High findings”
 - “Also check for logging and monitoring gaps”

The model revises the review based on updated context.

Considerations

The following considerations apply when using these prompts to perform security code review.

- **Verify CWE classifications.** The model suggests likely CWE mappings based on the vulnerability pattern. Verify these against the CWE specification before including them in formal documentation or compliance reports.
- **Treat fixes as starting points.** Code snippets provided by the model demonstrate the correct remediation approach but may require adaptation to your codebase’s conventions, error handling patterns, and testing requirements.
- **Provide project context for improved severity accuracy.** Without context about the deployment environment, the model states its assumptions. Providing details about network exposure, authentication requirements, and data sensitivity produces more accurate severity ratings.
- **Severity ratings may differ between personas.** The Security Engineer persona rates findings based strictly on attacker reachability (whether exploitation requires a separate prerequisite such as database compromise), while the SDE persona

emphasizes immediate remediation priority. The same vulnerability may receive different severity ratings depending on the persona used.

- **Review scope is limited to provided code.** The model does not speculate about vulnerabilities in code it cannot see. If a vulnerability spans multiple files, provide all relevant files in the same request.
- **Mitigate prompt injection risk.** Source code provided as input can contain instructions intended to manipulate the model. Treat code under review as untrusted data rather than as trusted instructions. For more information, see [Prompt injection \(https://docs.aws.amazon.com/bedrock/latest/userguide/prompt-injection.html\)](https://docs.aws.amazon.com/bedrock/latest/userguide/prompt-injection.html) .